数据结构与算法B

06-二叉树

北京大学

目录

- 6.1 树和二叉树基本概念
- 6.2 二叉树的存储结构
- 6.3 二叉树的遍历算法
- 6.4 二叉堆

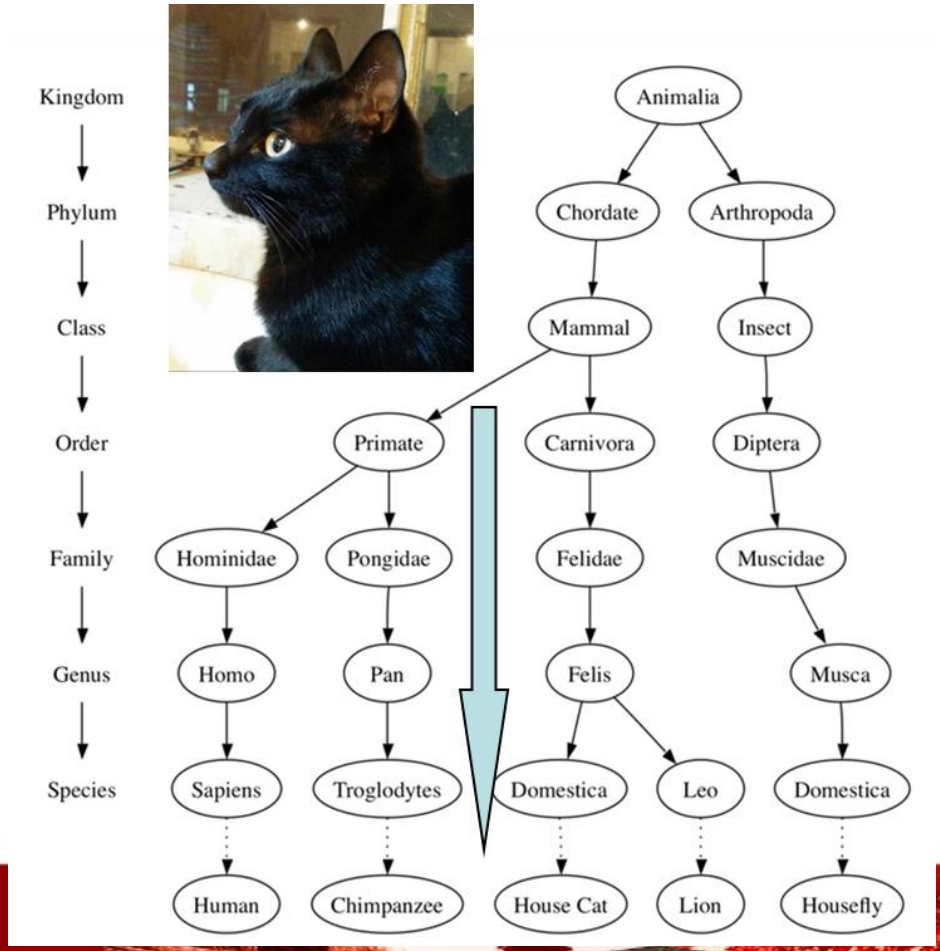
6.1 树和二叉树的基本概念

树的例子

- 本章我们来讨论一种基本的“非线性”数据结构——树；
- 树在计算机科学的各个领域中被广泛应用
  - 操作系统、图形学、数据库管理系统、计算机网络
- 跟自然界中的树一样，数据结构树也分为：根、枝和叶等三个部分
  - 但一般数据结构的图示把根放在上方
  - 叶放在下方

树的例子：生物学物种分类体系

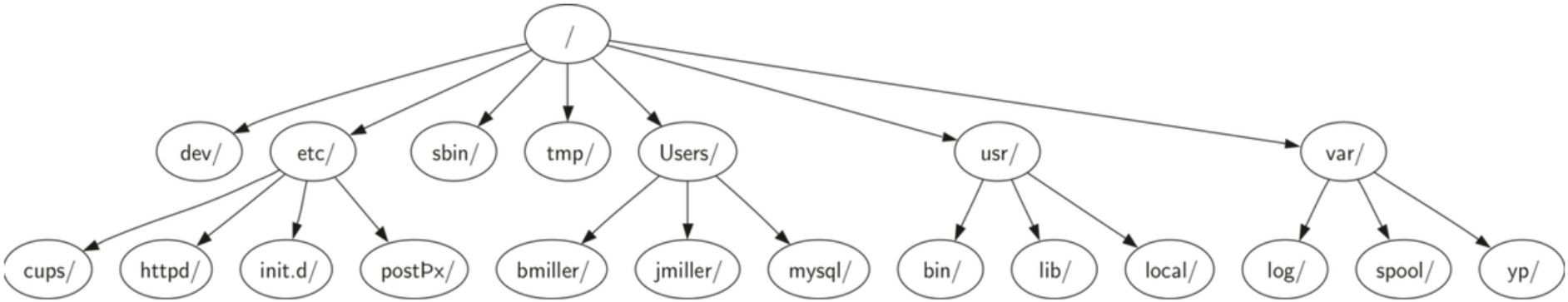
- 首先我们看到分类体系是层次化的
  - 树是一种分层结构
  - 越接近顶部的层越普遍
  - 越接近底部的层越独特
  - 界、门、纲、目、科、属、种
- 分类树的用法：辨认物种
  - 从顶端开始，沿着箭头方向向下
  - 门：脊索动物还是节肢动物？
  - 纲：哺乳动物么？
  - 目：食肉动物？
  - 科：猫科？
  - 属：猫属？
  - 种：家猫！



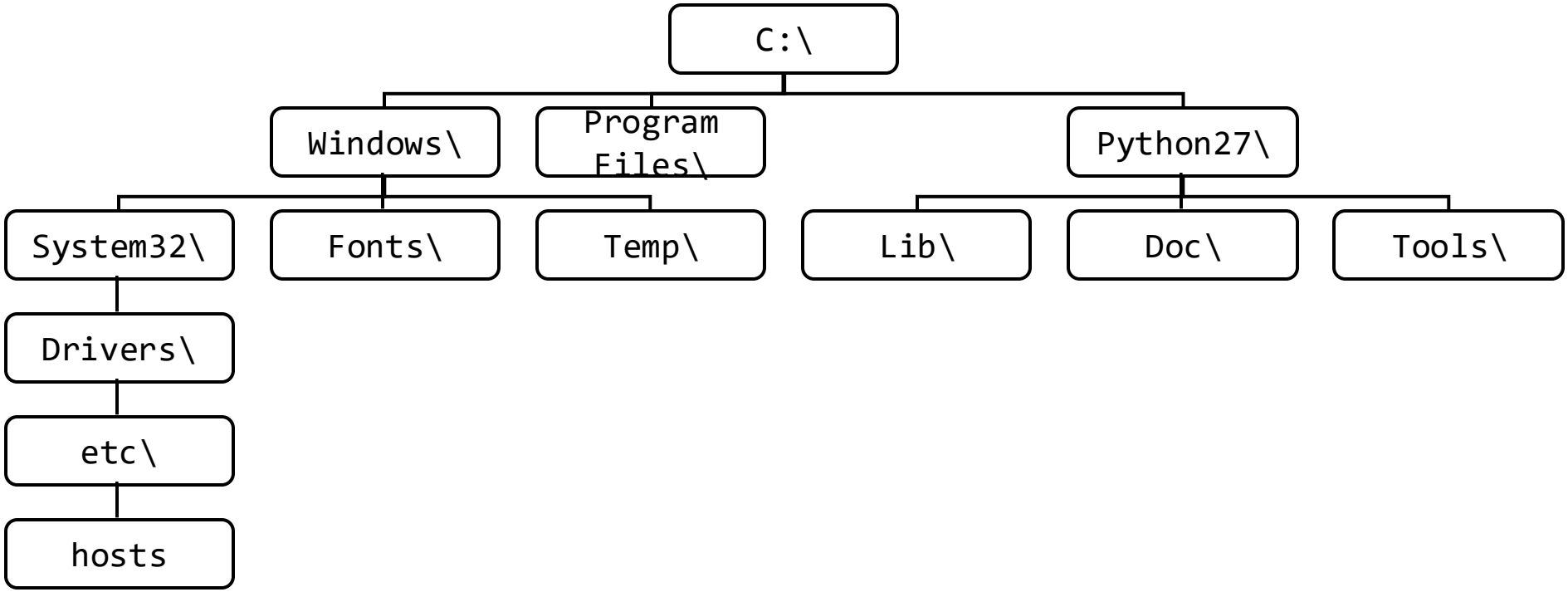
树的例子：生物学物种分类体系

- 分类树的第二个特征：一个节点的子节点与另一个节点的子节点相互之间是隔离、独立的
  - 猫属Felis和蝇属Musca下面都有Domestica的同名节点
  - 但相互之间并无任何关联，可以修改其中一个Domestica而不影响另一个。
- 分类树的第三个特征：每一个叶节点都具有唯一性
  - 可以用从根开始到达每个种的完全路径来唯一标识每个物种
  - 动物界->脊索门->哺乳纲->食肉目->猫科->猫属->家猫种
  - Animalia->Chordate->Mammal->Carnivora->Felidae->Felis->Domestica

树的例子：文件系统：Linux

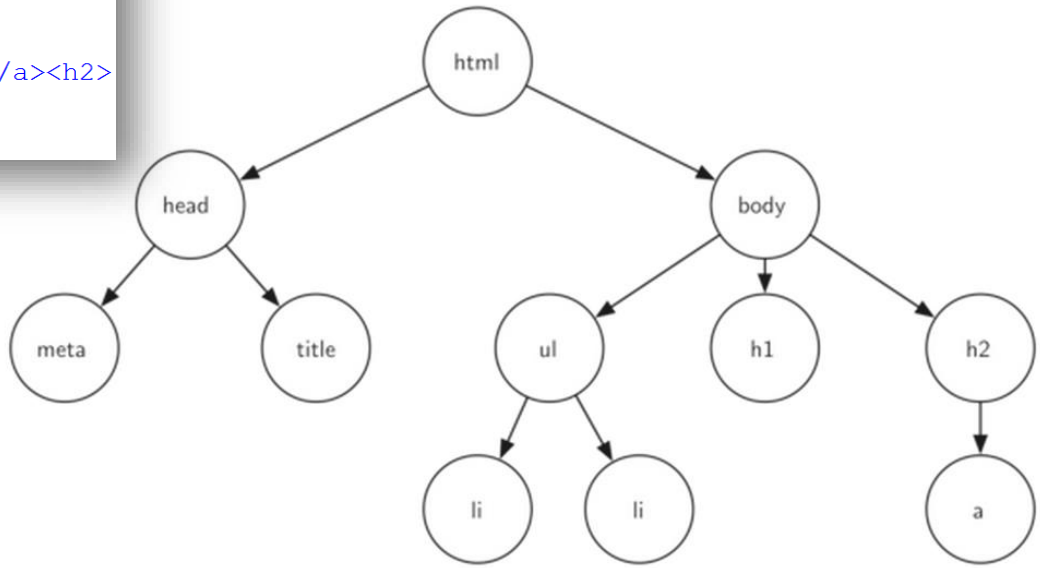


树的例子：文件系统：Windows

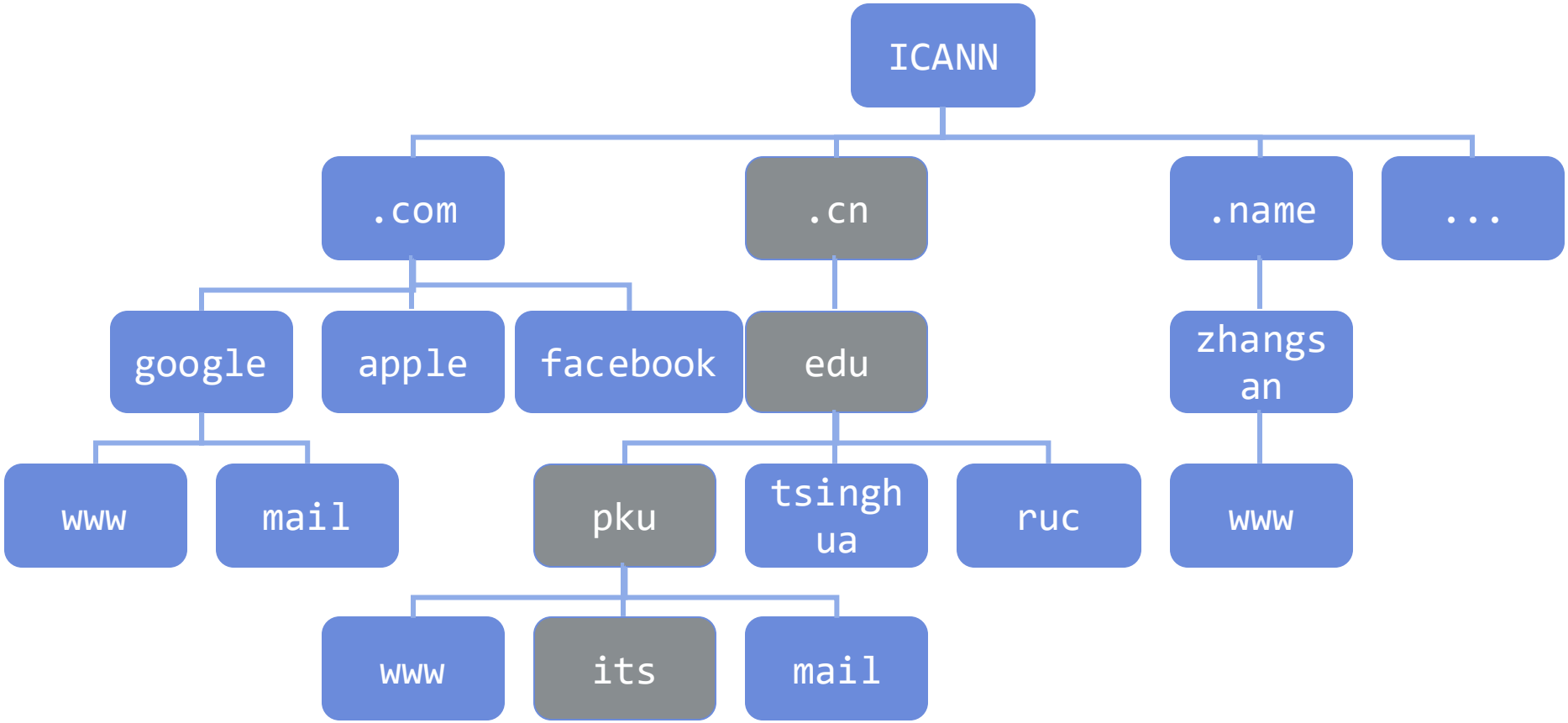


树的例子：HTML文档（嵌套标记）

```
<html xmlns="http://www.w3.org/1999/xhtml"
      xml:lang="en" lang="en">
<head>
    <meta http-equiv="Content-Type"
          content="text/html; charset=utf-8" />
    <title>simple</title>
</head>
<body>
<h1>A simple web page</h1>
<ul>
    <li>List item one</li>
    <li>List item two</li>
</ul>
<h2><a href="http://www.cs.luther.edu">Luther CS </a><h2>
</body>
</html>
```

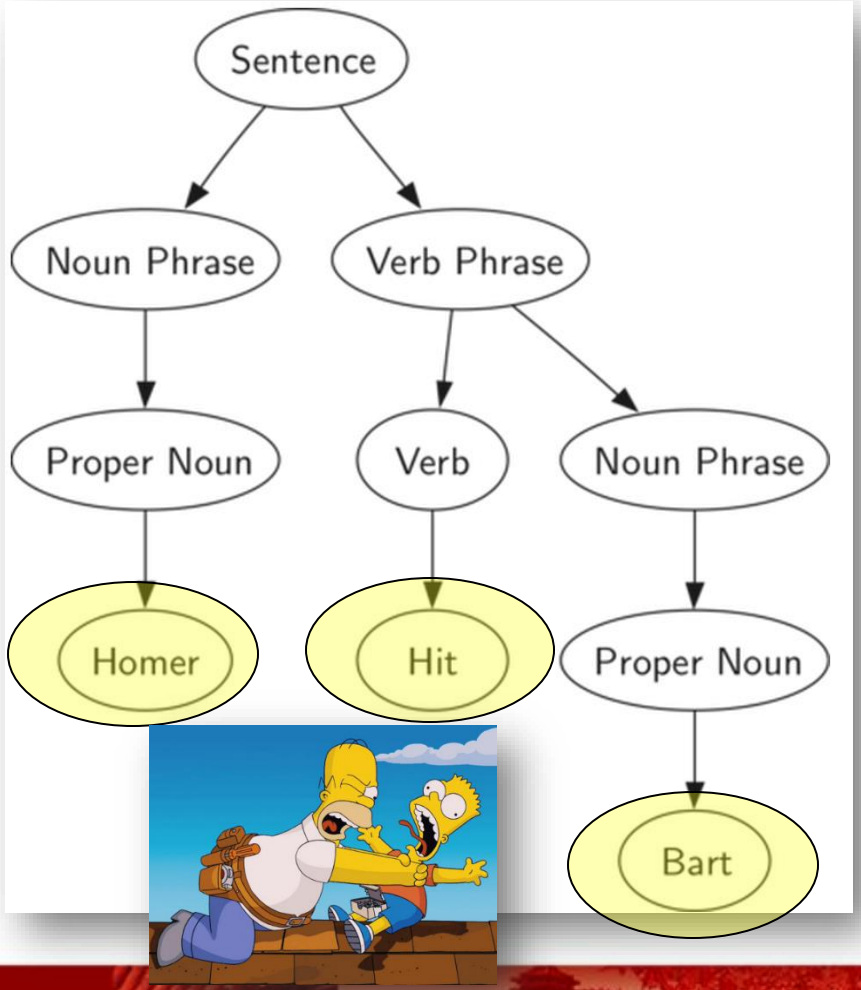


树的例子：域名体系


ICANN
.com
.cn
.name
...
google
apple
facebook
edu
zhangsan
www
mail
pku
tsinghua
ruc
www
www
its
mail

树的应用：解析树Parse Tree（语法树）

- 将树用于表示语言中句子，可以分析句子的各种语法成分，对句子的各种成分进行处理
- 语法分析树
  - 主谓宾，定状补
- 程序设计语言的编译
  - 词法、语法检查
  - 从语法树生成目标代码
- 自然语言处理
  - 机器翻译、语义理解



树的应用：解析树Parse Tree（表达式树）

- 我们还可以将表达式表示为树结构
  - 叶节点保存操作数，内部节点保存操作符
- 全括号表达式((7+3)*(5-2))
  - 由于括号的存在，需要计算*的话，就必须先计算7+3和5-2
  - 表达式树的层次帮助我们了解表达式计算的优先级
  - 越底层的表达式，优先级越高
- 树中每个子树都表示一个子表达式
  - 将子树替换为子表达式值的节点，即可实现求值

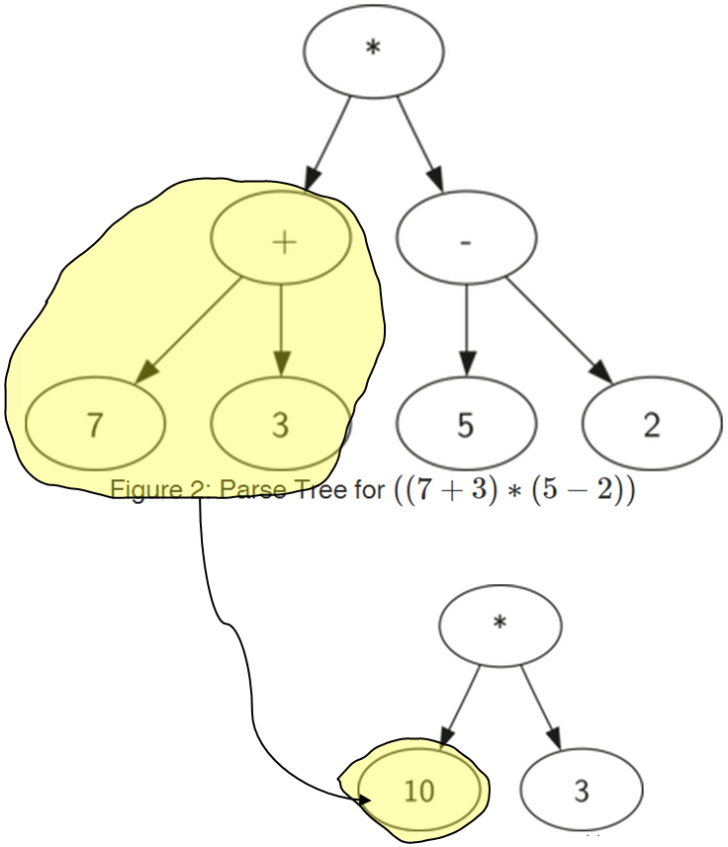


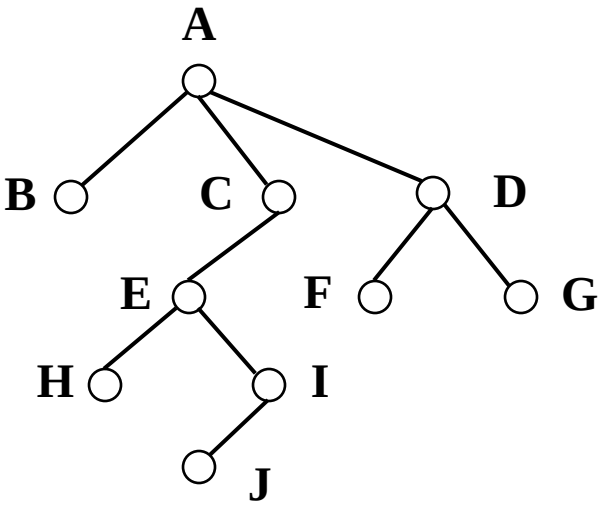
Figure 2: Parse Tree for $((7+3)*(5-2))$

树的逻辑结构

- 包含n个节点的有穷集合K (n>0)，且在K上定义了一个关系r，关系 r 满足以下条件：
  - 有且仅有一个节点$k_0 \in K$，它对于关系 r 来说没有前驱。节点k_0称作树的根；
  - 除节点k_0外，K中的每个节点对于关系 r 来说都有且仅有一个前驱；

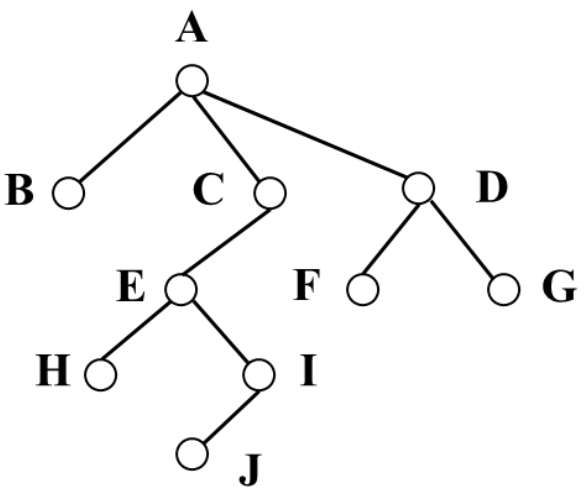
树的逻辑表示

- T = (N, R)
  - –结点集合 N = { A, B, C, D, E, F, G, H, I, J}
  - –N上的关系 R = { <A, B>, <A, C>, <A, D>, <C, E>, <D, F>, <D, G>, <E, H>, <E, I>, <I, J> }



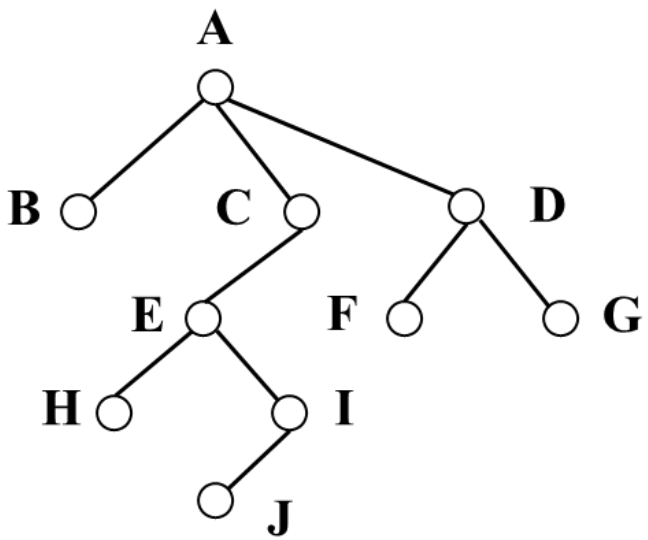
基本术语

- 父节点、子节点、边
  - 节点y是节点x的一棵子树的根，则x称作y的“父节点”（或父母）；
  - y称作x的“子节点”（或子女）；
  - 有序对<x，y>称作从x到y的“边”
- 兄弟节点
  - 具有同一父母的节点彼此称作“兄弟节点”
  - 树t中B，C，D互为兄弟节点，F，G互为兄弟节点，等等。
  - 注意，E和F并不是兄弟节点



基本术语

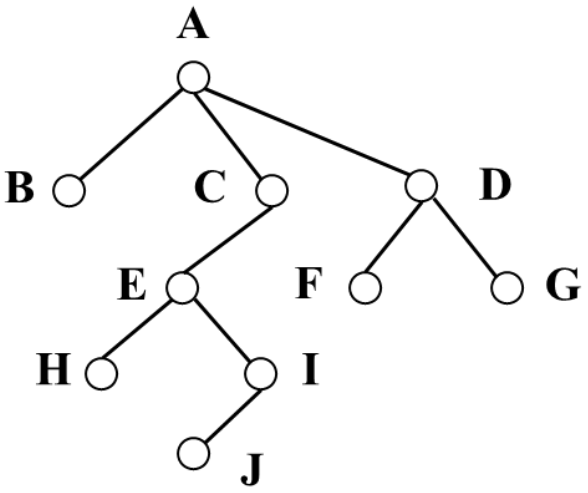
- 祖先、子孙
  - 若节点y在以节点x为根的一个子树（或树）中，且y≠x，称x是y的“祖先”，y是x的“子孙”



- 路径、路径长度
  - 对树上任意两个节点V_i，V_j，必然存在唯一的不重复的节点序列$\{V_i, P_1, P_2, \ldots, P_m, V_j\}$，使得$V_i$和$P_1$之间有边，$P_m$和$V_j$之间有边，$P_k$和$P_{k+1}$之间有边，这个节点序列就被称作为$V_i$到$V_j$的**路径**。
  - 路径上边的数量被称作是**路径长度**。

基本术语

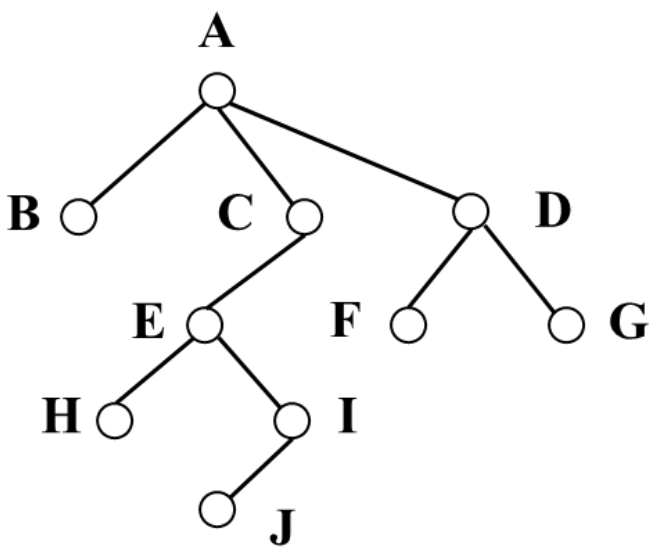
- 节点的层数、树的高度
  - **层数**：根为第 0 层
    - 其他节点的层数等于其父节点的层数加 1
  - **深度/高度：层数最大的叶节点的层数**
    - 也有些定义是上面的数值+1（取决于是否把根节点算上）



基本术语

节点的度数、树的度数

–节点的子女个数叫作节点的“度数”。树中度数最大的节点的度数叫作“树的度数”



树叶、分支节点

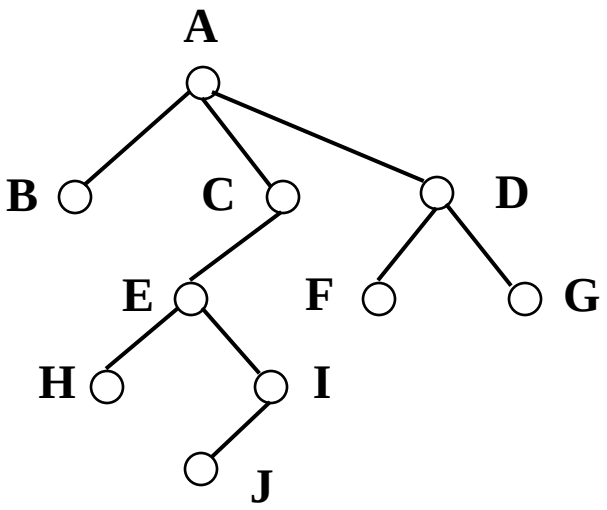
–度数为0的节点称作“树叶”（又叫终端节点）

–度数大于0的节点称作“分支节点”或非终端节点

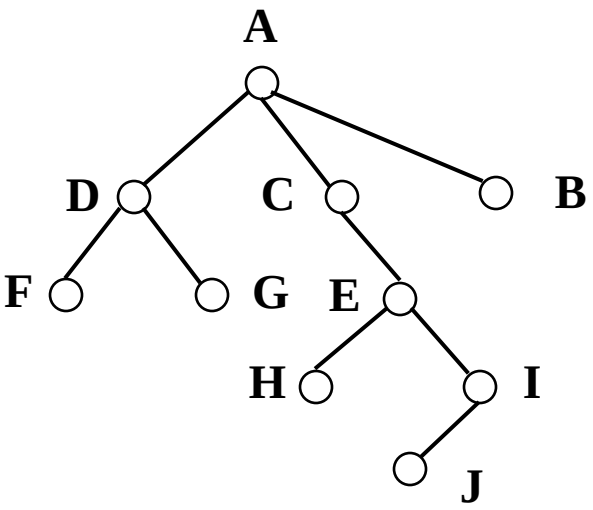
❖注意，节点的度数为1时，虽然只有一个子女，也叫分支节点。这两个术语对于根节点也不例外

基本术语

- 无序树、有序树
  - 对子树的次序不加区别的树叫作“无序树”。对子树之间的次序加以区别的树叫作“有序树”
  - 例如在下图中，按无序树的概念t和t’是同一棵树，按有序树的概念则是不同的树，**本章讨论的树一般是有序树**



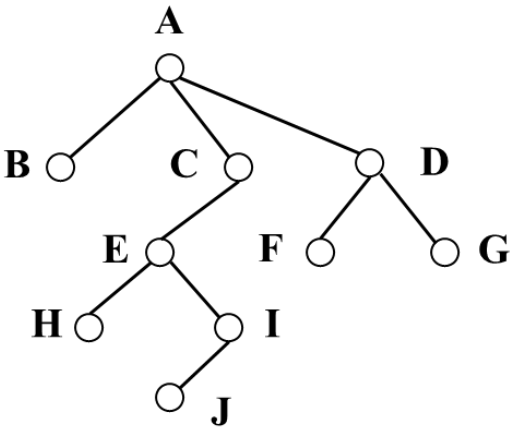
树t



树t’

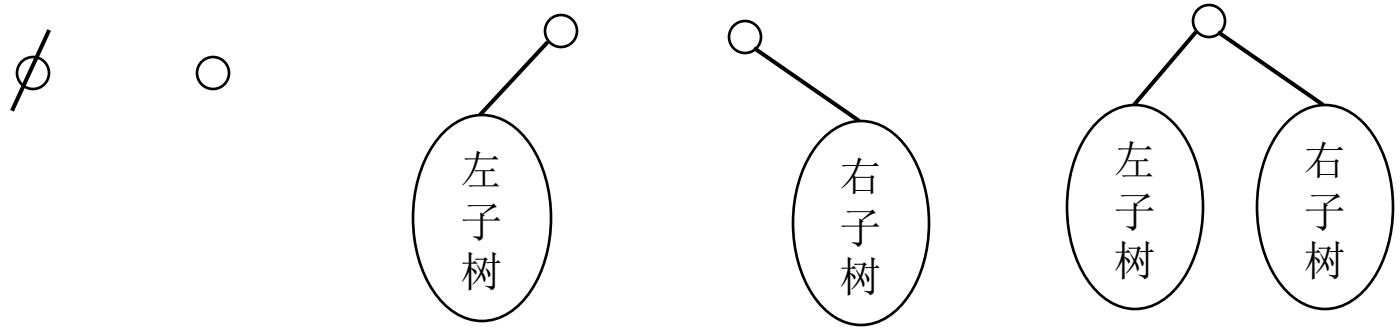
基本术语

- 节点的次序
  - 在有序树中可以从左到右地规定节点的次序
  - 例如下图中，节点B，C，D是从左到右排序的；可以说节点C是节点B右边的节点，是节点D左边的节点
  - 节点C的所有子女都在节点B及其子女的右边，而在节点D及其子女的左边
  - 按从左到右的顺序，可以把一个节点的子节点中最左边的节点简称“长子”，长子右边的节点称为“次子”，…



二叉树

- 递归定义：
  - 节点的有限集合，这个集合或者为**空集**，或者由一个**根**及两棵不相交的分别称作这个根的“**左子树**”和“**右子树**”的二叉树组成。
- 二叉树的五种基本形态



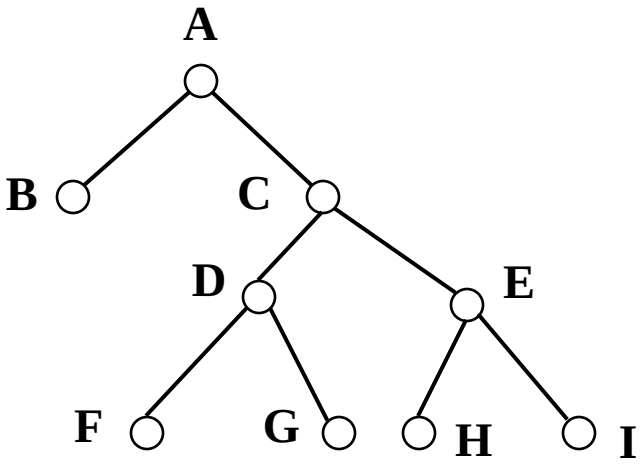
(a) (b) (c) (d) (e)

二叉树 vs 树

- 二叉树的术语与树中的相应部分类似
- 二叉树不是树的特殊情形，它们是两种数据结构
- 树和二叉树之间最主要的差别：
  - 二叉树中节点的子树要区分为左子树和右子树，即使在节点只有一棵子树的情况下也要明确指出该子树是左子树还是右子树
  - 譬如上例中，（c）和（d）是两棵不同的二叉树，但作为树，它们是相同的
- 度为 2 的有序树并不是二叉树
  - 第一子节点被删除，第二子节点就成为第一子节点

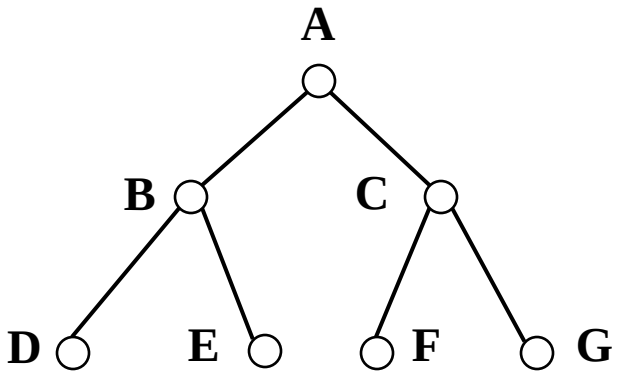
满二叉树（Full Binary Tree）

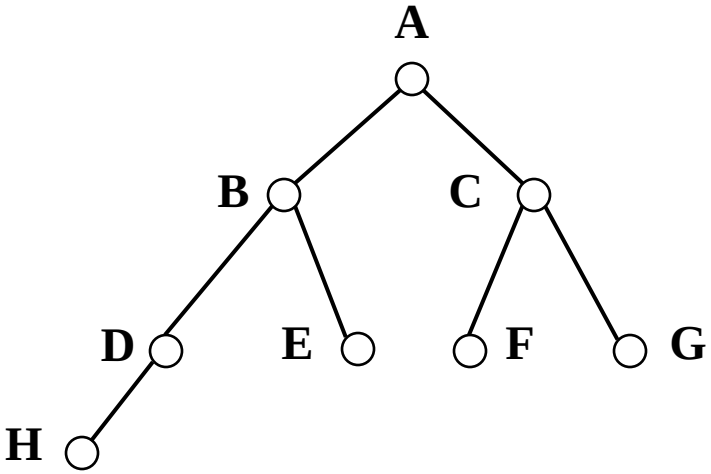
- 如果一棵二叉树的任何节点，或者是树叶，或者恰有两棵非空子树，则此二叉树称作满二叉树



完全二叉树（Complete Binary Tree）

- 若一棵二叉树，最多只有最下面的两层节点度数可以小于2，且最下面一层的节点都集中在该层最左边的若干位置上，则称此二叉树为完全二叉树

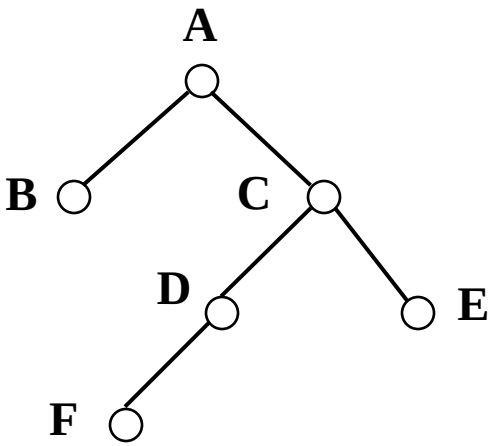


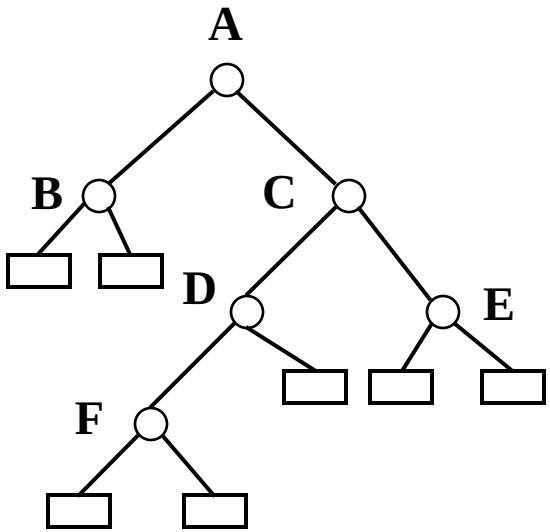


- 在许多算法和算法分析中都明显地或隐含地用到完全二叉树的概念

扩充二叉树（Extended Binary Tree）

- 扩充二叉树：把原二叉树的节点都变为度数为2的分支节点
  - 如果原节点的度数为2，则不变
  - 度数为1，则增加一个分支
  - 度数为0（树叶）增加两个分支





=>满二叉树

扩充二叉树

- 在扩充的二叉树里，新增加的节点（树叶），称为外部节点；原有节点称为内部节点
- 外部路径长度 E：在扩充的二叉树里从根到每个外部节点的路径长度之和
- 内部路径长度 I：在扩充的二叉树里从根到每个内部节点的路径长度之和

二叉树的基本性质

- 性质1：在非空二叉树的第i层上至多有2^i个节点($i \geq 0$)

- 证明：用归纳法来证。

① 归纳基础：i=0时，显然$2^i = 2^0 = 1$是对的

② 归纳假设：假定对所有的j($0 \leq j \leq i$)，命题成立，即第j层上至多有2^j个节点

③ 归纳推理：下面证明当j=i+1时，命题也成立

由归纳假设可知：第i层上至多有2^i个节点。又由于二叉树中每个节点的度至多为2，所以第i+1层上的最大节点个数是第i层上最大节点个数的2倍，即$2*2^i = 2^{i+1}$

结论：故二叉树的第i层上至多有2^i个节点($i \geq 0$)

二叉树的基本性质

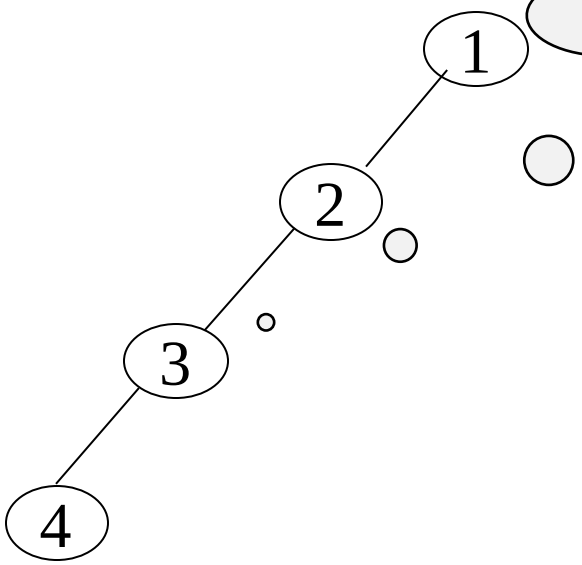
- 性质2：深度为k的二叉树中最多有2^{k+1} - 1个节点($k \geq 0$)

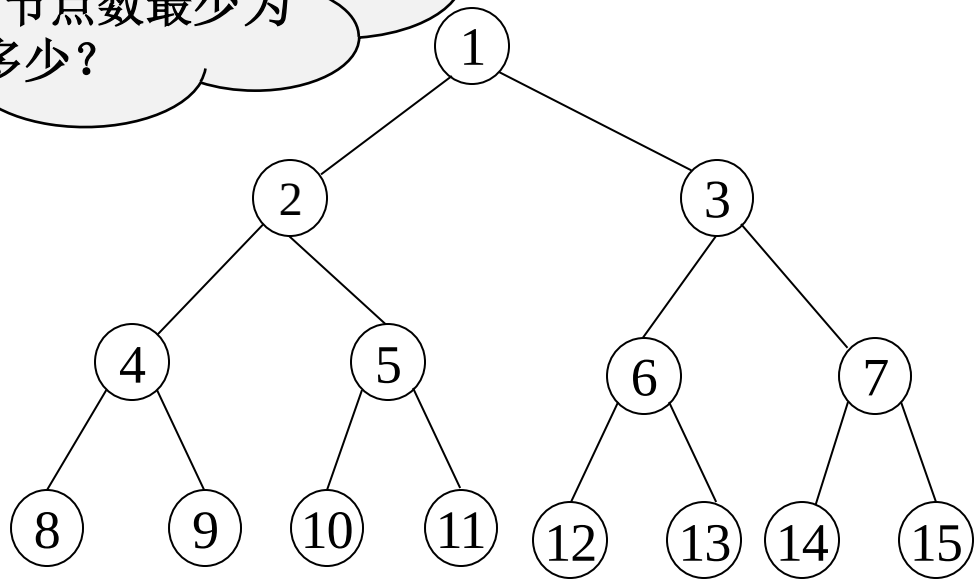
- 证明：假设第i层上的最大节点个数是m_i，由性质1可知，深度为k的二叉树中最大节点个数M为:

$$M = \sum_{i=0}^{k} m_i \leq \sum_{i=0}^{k} 2^i = 2^{k+1} - 1$$

示例

深度为h的二叉树上只有度为0和2的节点，那么二叉树节点数最少为多少？





节点最少的二叉树：

1+1+1+1

节点最多的二叉树：

$2^0+2^1+2^2+2^3$

二叉树的基本性质

- 性质3：对于任何一棵非空的二叉树，如果叶节点个数为n_0，度为2的节点个数为n_2，则有$n_0 = n_2 + 1$
- 证明：
  - 设非空二叉树中有n个节点，度为1的节点个数为n_1，则

$$n = n_0 + n_1 + n_2 \qquad (1)$$

  - 除根节点外，其余每个节点都有一个分支进入，假设B为分支总数，则

$$B = n - 1 \qquad (2)$$

  - 二叉树中的分支都是由度为1和2的节点发出的，有

$$B = n_1 + 2n_2 \qquad (3)$$

  - 综合(1)、(2)、(3)式可得

$$n_0 = n_2 + 1$$

二叉树的基本性质

- 性质4：具有n个节点的完全二叉树的深度k为 $\lfloor \log_2 n \rfloor$

证明：$n = 2^0 + 2^1 + 2^2 + \ldots + 2^{k-1} + m_k$

$= 2^k - 1 + m_k$

$2^k - 1 < n \leq 2^{k+1} - 1$ （性质2）

$2^k \leq n < 2^{k+1}$

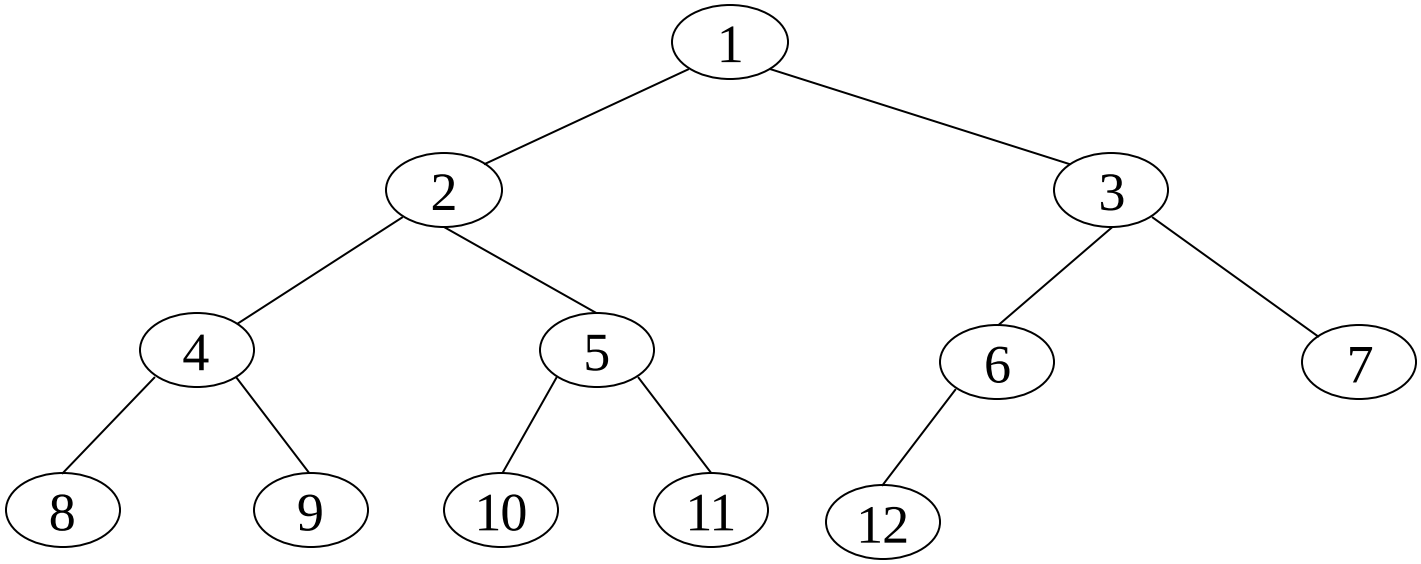
$k \leq \log_2 n < k+1$

$\therefore \quad k = \lfloor \log_2 n \rfloor$

示例

- 完全二叉树层次序列反映出它的结构

1 2 3 4 5 6 7 8 9 10 11 12



二叉树的基本性质

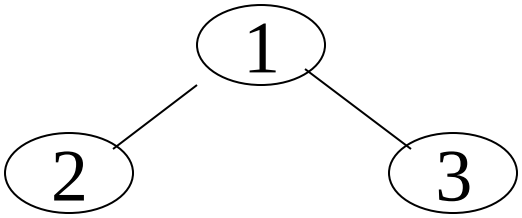
- 性质5：对于具有n个节点的完全二叉树，如果按照从上到下和从左到右的顺序对树中的所有节点从1开始进行编号，则对于任意的序号为 i 的节点，有：
  - 如果i＞1，则其父节点的序号为 i // 2；
    如果i = 1，则其是根节点，它没有父节点。
  - 如果2i≤n，则其左子节点的序号为2i；
    如果2i＞n，则其没有左子节点。
  - 如果2i+1≤n，则其右子节点的序号为2i+1；
    如果2i+1＞n，则其没有右子节点。

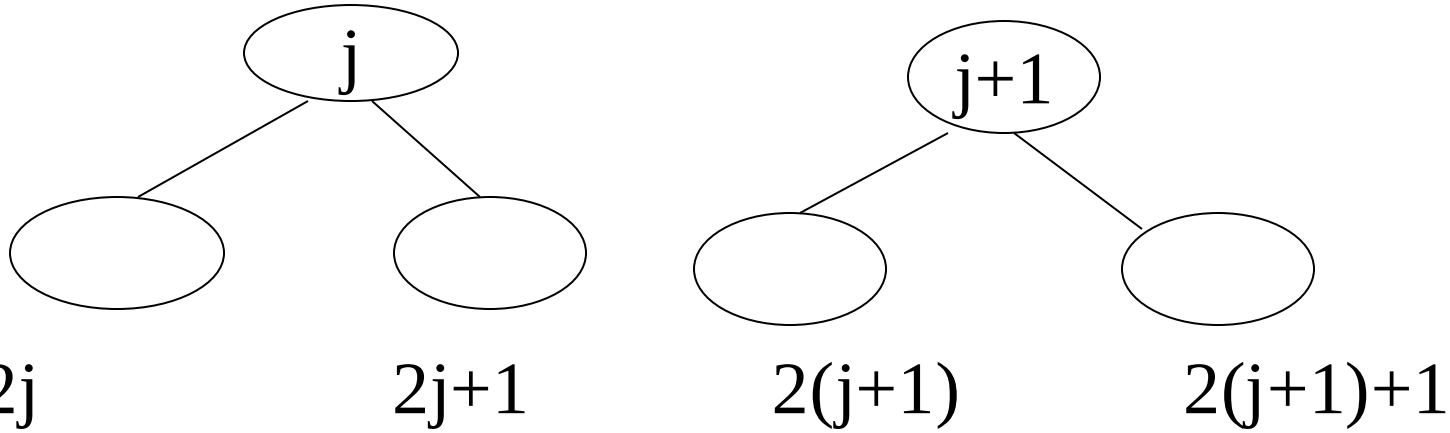
二叉树的基本性质

1. 归纳基础：对于（2）和（3）当i=1时，若$2i = 2 \le n$，左子女节点的序号为2。$2i+1 = 3 \le n$，右子女节点的序号为3。
2. 归纳假设：假设对于序号为j的节点，命题成立。
3. 归纳推理：对于i=j+1，
   - 其左子女节点的序号等于j的右子女节点的序号加1，即 $2j+1+1=2(j+1)$
   - 其右子女节点的序号等于：$2(j+1)+1$。
4. 根据（2）和（3），可知i的父节点的序号为$\lfloor i/2 \rfloor$.

结论：完全二叉树的层次序列，反映了它的结构。

性质5的示例





二叉树的基本性质

- 性质6：在满二叉树中，叶节点个数比分支节点个数多1。

- 实际上是性质3的推论
  - 性质3：对于任何一棵非空的二叉树，如果叶节点个数为n_0，度为2的节点个数为n_2，则有$n_0 = n_2 + 1$
  - 叶节点即度为0的节点，分支节点即度为2的节点

二叉树的基本性质

- 性质7：在扩充的二叉树里，新增加的外部节点的个数比原来的内部节点个数多1。

证明

- 对于任何一棵非空的二叉树，如果叶节点个数为n_0，度为2的节点个数为n_2，度为1的节点个数为n_1，原来的内部节点个数为$n = n_0 + n_1 + n_2$,因为$n_0 = n_2 + 1$，则$n = n_1 + 2n_0 - 1$
- 新增加的外部节点为：$m = n_1 + 2n_0$

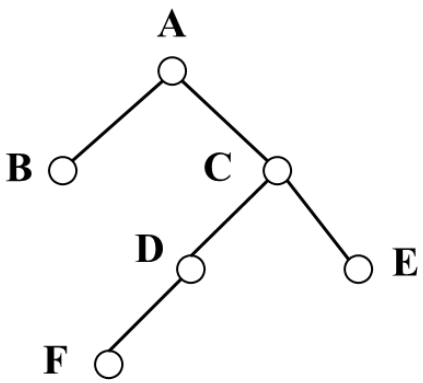
二叉树的基本性质

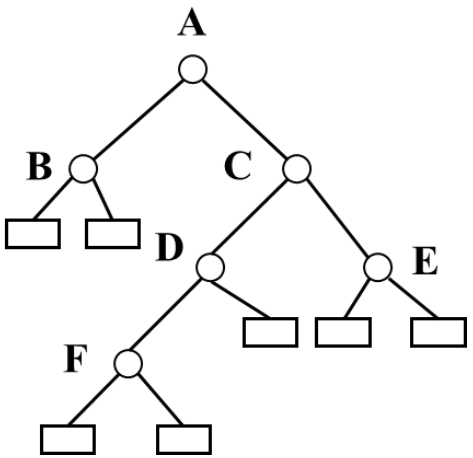
- 性质8：对任意扩充二叉树，E 和 I 之间满足：E = I + 2n，其中n是内部节点个数。

譬如

E = 2 + 2 + 4 + 4 + 3 + 3 + 3 = 21

I = 0 + 1 + 1 + 2 + 2 + 3 = 9





证明

1. 归纳基础：当n=1时，I=0, E=2，此等式成立。
2. 归纳假设：设有n个内部节点的扩充二叉树，下式成立。

$$E_n = I_n+2n \qquad (1)$$

3. 归纳推理：对于 n+1 个内部节点的扩充二叉树，去掉一个原来为树叶、路径长度为K的内部节点，内部路径长度变为：

$$I_n = I_{n+1}-K \qquad (2)$$

外部路径长度变为：

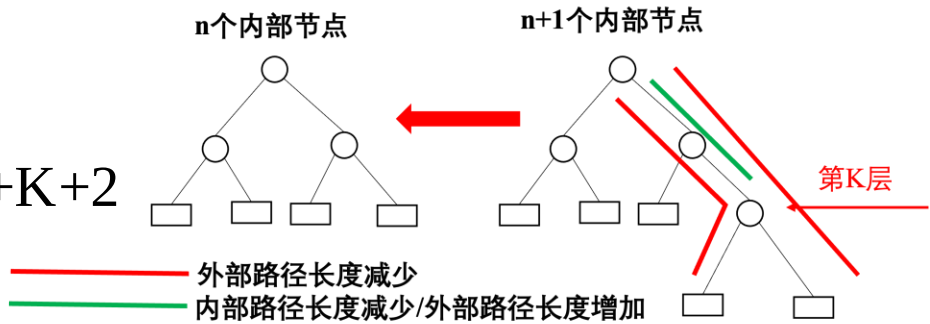
$$E_n = E_{n+1}-2(K+1)+K = E_{n+1} -K-2$$

即：　$E_{n+1}= E_n+K+2$

$$E_{n+1} = \underline{(I_n+2n)} +K+2 = \underline{(I_{n+1}-K)} +2n+K+2$$

代入（1）　　代入（2）

$$= I_{n+1}+2(n+1)$$



6.2 二叉树的存储结构

二叉树的实现

- 根据应用的不同以及二叉树本身的不同特点，二叉树可采用不同的存储结构
  - 顺序表示
  - 链式表示

完全二叉树的顺序表示

- 完全二叉树节点的顺序存储
  - 所有节点按层次顺序依次存储在连续的存储单元中，
  - 根据一个节点的存储地址就可算出它的左右子女，父母的存储地址，如同存储了相应的指针一样。
- 顺序表示是存储完全二叉树的最简、最节省空间的存储方式
  - 完全二叉树的顺序存储，在存储结构上是线性的，但在逻辑结构上它仍然是二叉树型结构

一般二叉树的顺序表示

- 增加空节点来构造一棵完全二叉树，再以二叉树的方式存储
- 接近于完全二叉树的形态，需要增加的空节点数目不多，则也可采用顺序表示；

	A	B	C	∧	D	E	∧	∧	∧	F	∧	∧	G
数组下标	0	1	2	3	4	5	6	7	8	9	10	11	12

北京大学

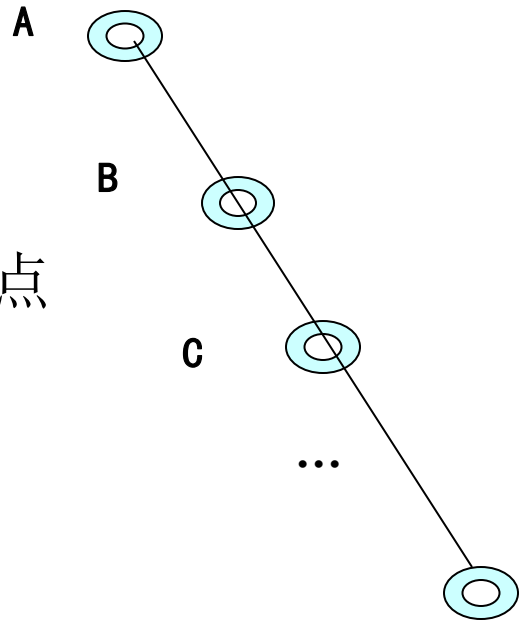
顺序表示的二叉树定义

采用顺序存储表示的二叉树定义如下:

```
class BinaryTree:
    def __init__(self, max_nodes=100):
        self.nodes = [None] * max_nodes
        self.size = 0

    def insert(self, value):
        if self.size < len(self.nodes):
            self.nodes[self.size] = value
            self.size += 1

    def parent(self, index):
        return self.nodes[(index - 1) // 2] if index > 0 else None

    def left_child(self, index):
        left_index = 2 * index + 1
        return self.nodes[left_index] if left_index < self.size else
None

    def right_child(self, index):
        right_index = 2 * index + 2
        return self.nodes[right_index] if right_index < self.size
else None
```

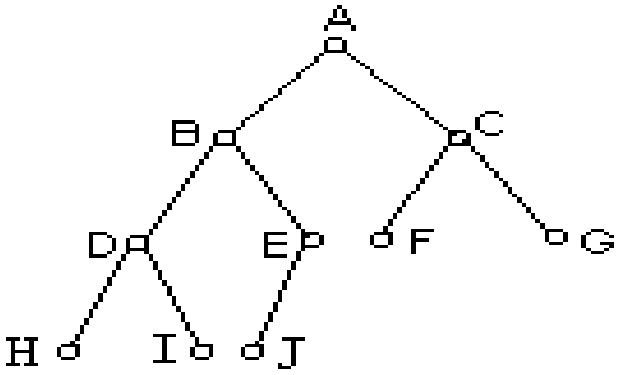
一般二叉树的顺序存储的问题

- 如果需要增加很多空节点才能将一棵一般的二叉树改造成完全二叉树，那么采用顺序表示就会造成空间的大量浪费，这时就不适合用顺序表示
- 对于一种特殊情况，如果二叉树为右单支树，其深度为k（有k+1个节点]，则需要一个长度为2^{k+1}-1 的一维数组，造成(2^{k+1}-1)-(k+1)个节点浪费。
  - ✓如果k=5, 则有57个节点空间浪费



完全二叉树的顺序表示

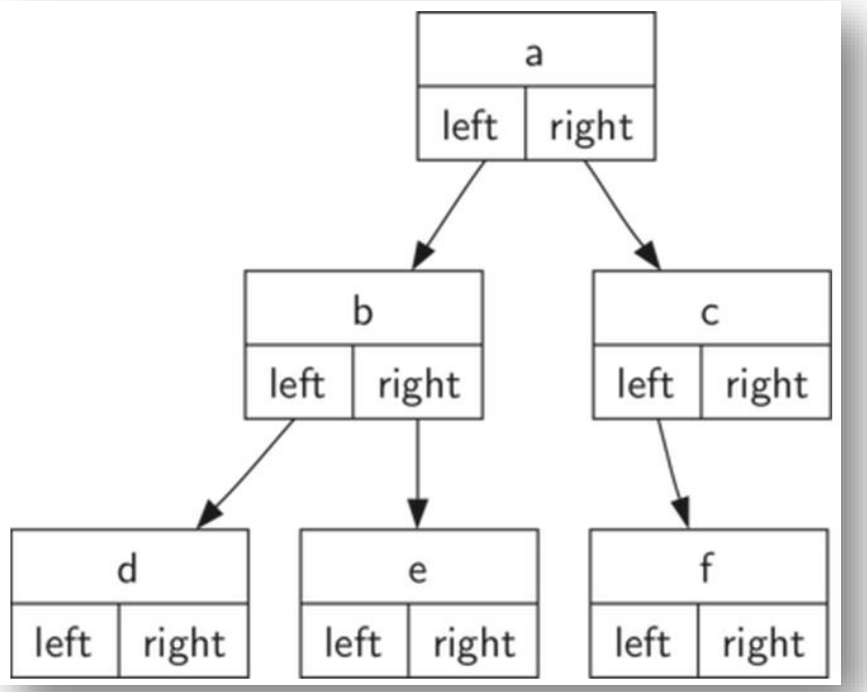
- 用一组连续的存储单元来存放二叉树中的节点
  - 完全二叉树中节点的层次序号可以反映出节点之间的逻辑关系
  - 例：完全二叉树的顺序表示（性质5）



	A	B	C	D	E	F	G	H	I	J
数组下标	0	1	2	3	4	5	6	7	8	9

二叉树的链式存储结构

- 同样可以用类似链表的节点链接法来实现树
  - 每个节点保存根节点的数据项，以及指向左右子树的链接
- 定义一个BinaryTree类
  - 成员key保存根节点数据项
  - 成员left_child/right_child则保存指向左/右子树的引用（同样是BinaryTree对象）




```python
class BinaryTree:
    def __init__(self, rootobj):
        self.key = rootobj
        self.left_child = None
        self.right_child = None
```

二叉树的链式存储结构

- insert_left/right方法
  - 过程与线性表的节点插入相似

```
r = BinaryTree('a')
r.insert_left('b')
r.insert_right('c')
r.get_right_child().set_root_val('hello')
r.get_left_child().insert_right('d')
```

```
def insert_left(self, new_node):
    if self.left_child == None:
        self.left_child = BinaryTree(new_node)
    else:
        t = BinaryTree(new_node)
        t.left_child = self.left_child
        self.left_child = t

def insert_right(self, new_node):
    if self.right_child == None:
        self.right_child = BinaryTree(new_node)
    else:
        t = BinaryTree(new_node)
        t.right_child = self.right_child
        self.right_child = t

def get_right_child(self):
    return self.right_child

def get_left_child(self):
    return self.left_child

def set_root_val(self, obj):
    self.key = obj

def get_root_val(self):
    return self.key
```

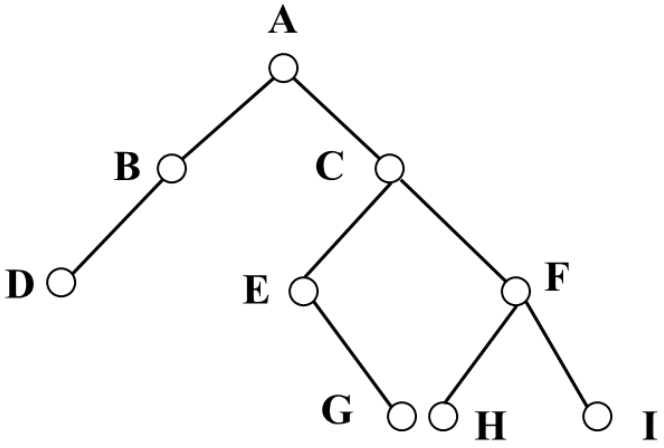
6.3 二叉树的遍历算法

二叉树的遍历概念

- 二叉树的遍历是指按某种方式访问二叉树中的所有节点，使每个节点被访问一次且只被访问一次
  - 遍历二叉树的过程实际上就是把二叉树的节点放入一个线性序列的过程，或者说是把二叉树进行线性化
- 按照遍历的顺序，可以将遍历方式分成：
  - 深度优先遍历（Depth-first traversal）
    - 按深度方向，尽可能深地访问节点，然后回溯
    - 按照访问根节点的时机，进一步分成：前序、中序、后序
  - 广度优先遍历（Breadth-first traversal）
    - 按层次顺序，逐层访问所有节点

前序遍历

- 若以符号D、L、R分别表示访问根节点、遍历根节点的左子树、遍历根节点的右子树。



- 前序次序：DLR
  (1) 访问根；
  (2) 按前序次序遍历左子树；
  (3) 按前序次序遍历右子树。

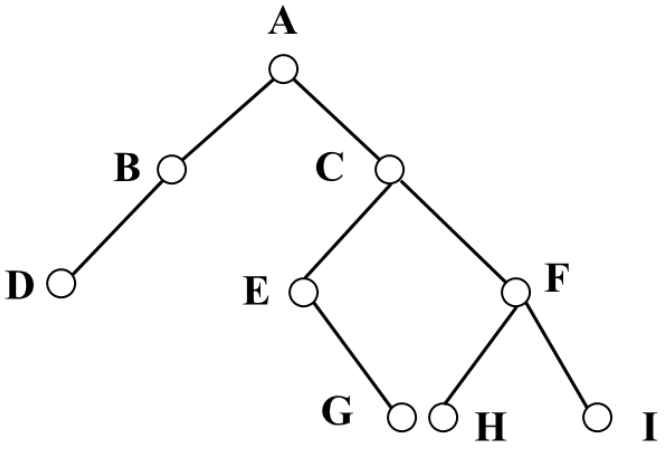
- 将按前序次序对一棵二叉树遍历得到的线性表称为这棵二叉树的前序序列。
  –此二叉树的前序序列为：A B D C E G F H I

二叉树前序次序遍历的递归算法

```
def pre_order(tree):
    if tree:
        print(tree.get_root_val())
        pre_order(tree.get_left_child())
        pre_order(tree.get_right_child())
```

中序遍历

- 中序次序(对称序法)：LDR
  (1) 按对称序次序遍历左子树;
  (2) 访问根;
  (3) 按对称序次序遍历右子树。



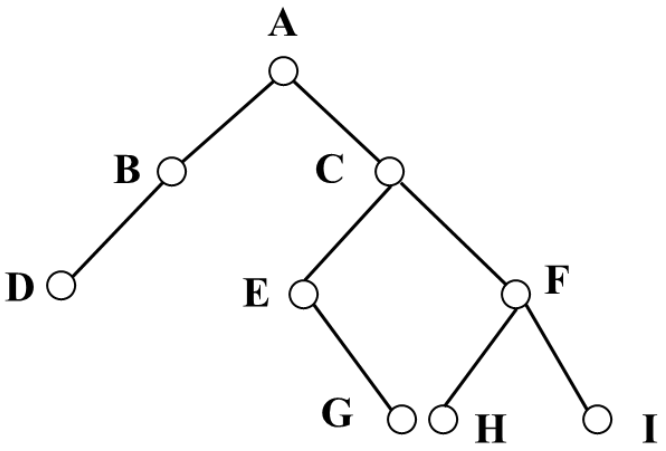
- 将按中序次序对一棵二叉树遍历得到的线性表称为这棵二叉树的中序序列。
  –中序(对称序)序列是： D B A E G C H F I

二叉树中序次序遍历的递归算法

```
def in_order(tree):
    if tree:
        in_order(tree.get_left_child())
        print(tree.get_root_val())
        in_order(tree.get_right_child())
```

后序遍历

- 后序次序：LRD
  (1) 按后序次序遍历左子树；
  (2) 按后序次序遍历右子树；
  (3) 访问根。



- 将按后序次序对一棵二叉树遍历得到的线性表称为这棵二叉树的后序序列。
  –此二叉树的后序序列是：D B G E H I F C A

二叉树后序次序遍历的递归算法

```
def post_order(tree):
    if tree:
        post_order(tree.get_left_child())
        post_order(tree.get_right_child())
        print(tree.get_root_val())
```

二叉树遍历的非递归算法

- 利用栈实现二叉树遍历的非递归算法。
  - 前序遍历
  - 中序遍历
  - 后序遍历

- 回忆：使用栈将任何递归程序转变为非递归程序
  - 模拟函数的调用栈

前序遍历的非递归算法

- 待模拟的递归函数

```
def pre_order(tree):
    if tree:
        print(tree.get_root_val())
        pre_order(tree.get_left_child())
        pre_order(tree.get_right_child())
```

- 在模拟函数调用链时，需要用栈存储的变量
  - 参数 tree
  - 函数执行进度：子函数返回时，定位从哪里继续执行

前序遍历的非递归算法

- 待模拟的递归函数

```
def pre_order(tree):
    if tree:
        print(tree.get_root_val())
        pre_order(tree.get_left_child())
        pre_order(tree.get_right_child())
```

flag=0

flag=1

flag=2

- 在模拟函数调用链时，需要用栈存储的变量
  - 参数 tree
  - 函数执行进度：由于调用了子函数，函数的递归执行过程并不是连续的。整个执行过程被分为若干部分，
    - 初次调用(flag=0)、第一次子函数返回后(flag=1)、第二次子函数返回后(flag=2)
    - 当执行权交给当前的调用时，据此确定从哪里开始执行

前序遍历的非递归算法

- 待模拟的递归函数

```
def pre_order(tree):
    if tree:
        print(tree.get_root_val())
        pre_order(tree.get_left_child())
        pre_order(tree.get_right_child())
```

flag=0

flag=1

flag=2

- 非递归函数

```
def non_recursive_pre_order(tree):
    stack = []
    stack.append([tree, 0])
    while stack:                        # len(stack) > 0
        root, flag = stack[-1]
        if flag == 0:
            if root:
                print(root.get_root_val(), end=" ")
                stack[-1][1] += 1   # 调用子函数前，修改flag
                stack.append([root.get_left_child(), 0])
            else:
                stack.pop()         # 递归出口
        elif flag == 1:
            stack[-1][1] += 1       # 调用子函数前，修改flag
            stack.append([root.get_right_child(), 0])
        elif flag == 2:
            stack.pop()             # 执行完毕退出
```

北京大学

中序遍历的非递归算法

- 待模拟的递归函数

```
def in_order(tree):
    if tree:
        in_order(tree.get_left_child())
        print(tree.get_root_val())
        in_order(tree.get_right_child())
```

flag=0

flag=1

flag=2

- 非递归函数

访问根节点的时机变动

```
def non_recursive_in_order(tree):
    stack = []
    stack.append([tree, 0])
    while stack:                        # len(stack) > 0
        root, flag = stack[-1]
        if flag == 0:
            if root:
                stack[-1][1] += 1       # 调用子函数前，修改flag
                stack.append([root.get_left_child(), 0])
            else:
                stack.pop()             # 递归出口
        elif flag == 1:
            print(root.get_root_val(), end=" ")
            stack[-1][1] += 1           # 调用子函数前，修改flag
            stack.append([root.get_right_child(), 0])
        elif flag == 2:
            stack.pop()                 # 执行完毕退出
```

北京大学

后序遍历的非递归算法

- 待模拟的递归函数

```
def post_order(tree):
    if tree:
        post_order(tree.get_left_child())
        post_order(tree.get_right_child())
        print(tree.get_root_val())
```

flag=0

flag=1

flag=2

- 非递归函数

访问根节点的时机变动

```
def non_recursive_post_order(tree):
    stack = []
    stack.append([tree, 0])
    while stack:                        # len(stack) > 0
        root, flag = stack[-1]
        if flag == 0:
            if root:
                stack[-1][1] += 1       # 调用子函数前，修改flag
                stack.append([root.get_left_child(), 0])
            else:
                stack.pop()             # 递归出口
        elif flag == 1:
            stack[-1][1] += 1           # 调用子函数前，修改flag
            stack.append([root.get_right_child(), 0])
        elif flag == 2:
            print(root.get_root_val(), end=" ")
            stack.pop()                 # 执行完毕退出
```

北京大学

二叉树遍历的非递归算法

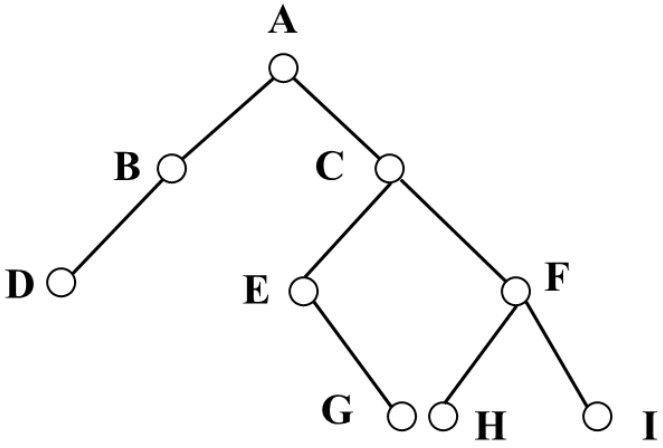
- 递归算法非常简洁——推荐使用
  - 当前的编译系统优化效率很不错了
- 特殊情况用栈模拟递归
  - 理解编译栈的工作原理
  - 理解深度优先遍历的回溯特点

深度优先遍历算法的复杂度分析

- 时间复杂度为O(n)，n为树中节点的数量
  - 遍历每个节点本身只需要O(1)的操作
- 空间复杂度为递归调用栈的深度，与树的高度有关
  - 最好情况下，树为完全二叉树，高为 O(log n)
  - 最坏情况下，树退化为线性结构，高为 O(n)

广度（宽度）优先遍历二叉树

- 从二叉树的第 0 层（根节点）开始，自上而下 逐层遍历；在同一层中，按照 从左到右 的顺序对节点逐一访问。
- 例如：A B C D E F G H I



广度（宽度）优先遍历二叉树

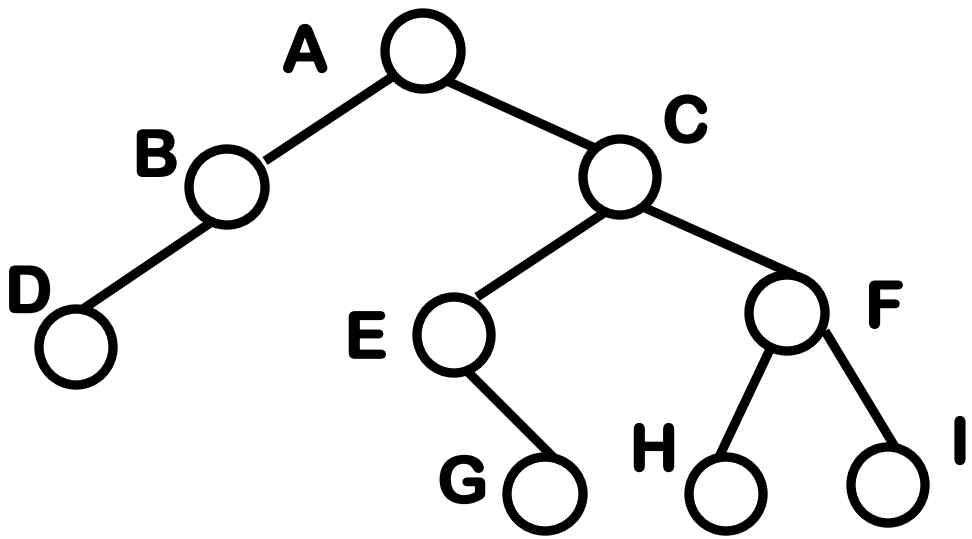
```python
from collections import deque

class BinaryTree:
    def __init__(self,rootobj):
        self.key = rootobj
        self.left_child = None
        self.right_child = None
    # 之前定义过的一些函数
    # ...
    # 广度优先遍历
    def bfs(self):
        queue = deque()
        queue.append(self)
        while len(queue) > 0:
            node = queue.popleft() # 将队列头节点弹出
            print(node.key) # 访问队列头节点
            if node.left_child:
                queue.append(node.left_child) # 将左子节点加入到队列中
            if node.right_child:
                queue.append(node.right_child) # 将右子节点加入到队列中
```

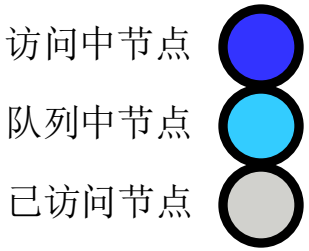
广度（宽度）优先遍历二叉树

BFS序列

队列

A	B	C	D	E	F	G	H	I

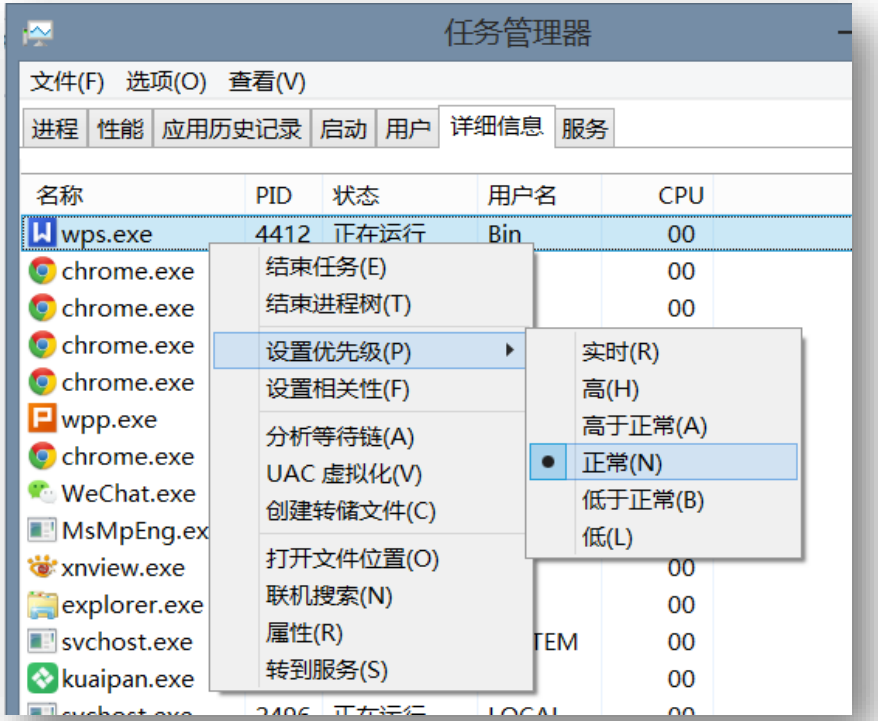




6.4 二叉堆

优先队列Priority Queue

- 在之前我们学习了一种FIFO数据结构队列queue
- 队列有一种变体称为“优先队列”。
  - 银行窗口取号，VIP客户优先级高，可以插到队首
  - 操作系统中执行关键任务的进程或用户特别指定进程在调度队列中靠前



优先队列Priority Queue

- 优先队列的出队dequeue操作跟队列一样，都是从队首出队；
- 但在优先队列内部，数据项的次序却是由“优先级”来确定：高优先级的数据项排在队首，而低优先级的数据项则排在后面。
  - 这样，优先队列的入队操作就比较复杂，需要将数据项根据其优先级尽量挤到队列前方。
- 思考：有什么方案可以用来实现优先队列？
  - 出队和入队的复杂度大概是多少？

二叉堆Binary Heap实现优先队列

- 实现优先队列的经典方案是采用二叉堆数据结构
  - 二叉堆能够将优先队列的入队和出队复杂度都保持在O(log n)
- 二叉堆的有趣之处在于，其逻辑结构上象二叉树，却是用非嵌套的列表来实现的！
- 最小key排在队首的称为“最小堆min heap”
  - 反之，最大key排在队首的是“最大堆max heap”

- ADT BinaryHeap的操作定义如下：
  - BinaryHeap()：创建一个空二叉堆对象；
  - insert(k)：将新key加入到堆中；
  - findMin()：返回堆中的最小项，最小项仍保留在堆中；
  - delMin()：返回堆中的最小项，同时从堆中删除；
  - isEmpty()：返回堆是否为空；
  - size()：返回堆中key的个数；
  - buildHeap(list)：从一个key列表创建新堆

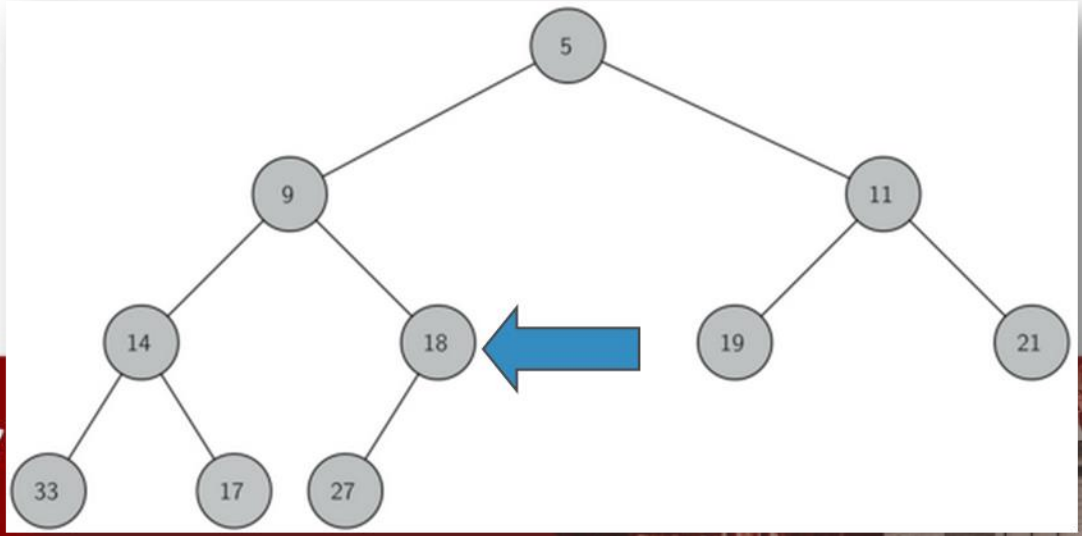
ADT BinaryHeap的操作示例

```python
from pythonds.trees.binheap import BinHeap

bh = BinHeap()
bh.insert(5)
bh.insert(7)
bh.insert(3)
bh.insert(11)

print(bh.delMin())
print(bh.delMin())
print(bh.delMin())
print(bh.delMin())
```

```
>>> ========
>>>
3
5
7
11
>>>
```

用完全二叉树实现堆

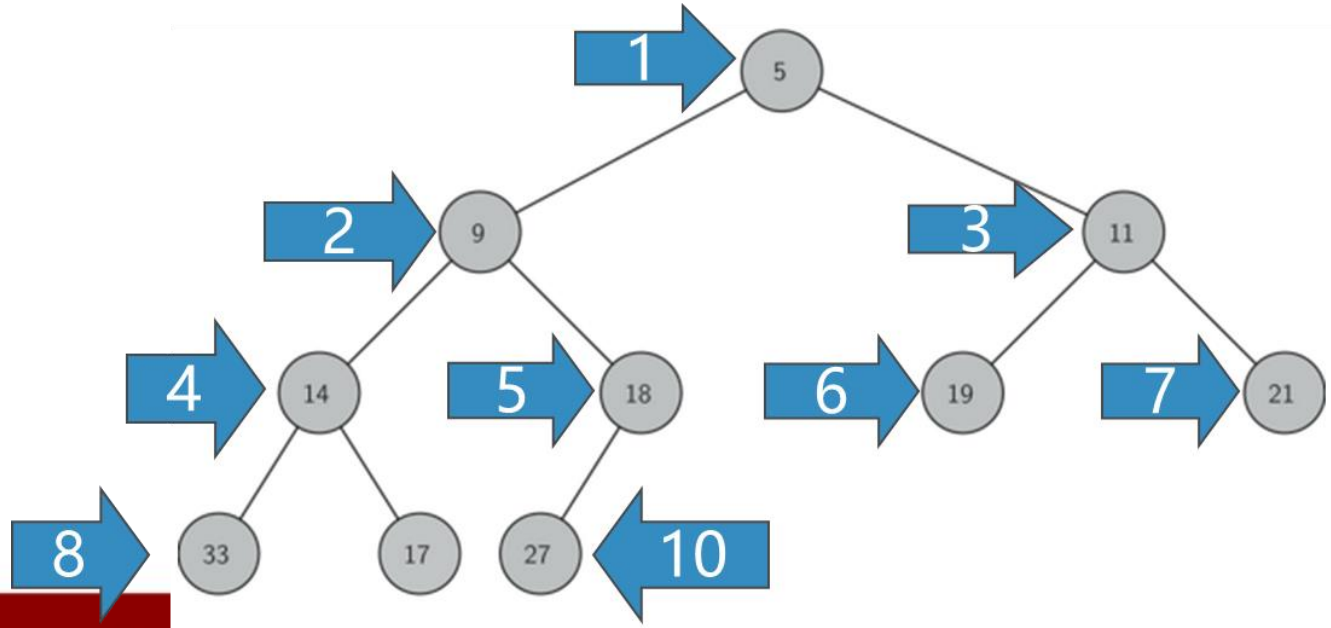
- 为了使堆操作能保持在对数水平上，必须采用二叉树结构；
- 同样，如果要使操作始终保持在对数数量级上，就必须始终保持二叉树的“平衡”——树根左右子树拥有相同数量的节点
- 我们采用“完全二叉树”的结构来近似实现“平衡”
  - 完全二叉树，叶节点最多只出现在最底层和次底层，而且最底层的叶节点都连续集中在最左边，每个内部节点都有两个子节点，最多可有1个节点例外



北京大学

完全二叉树的列表实现及性质

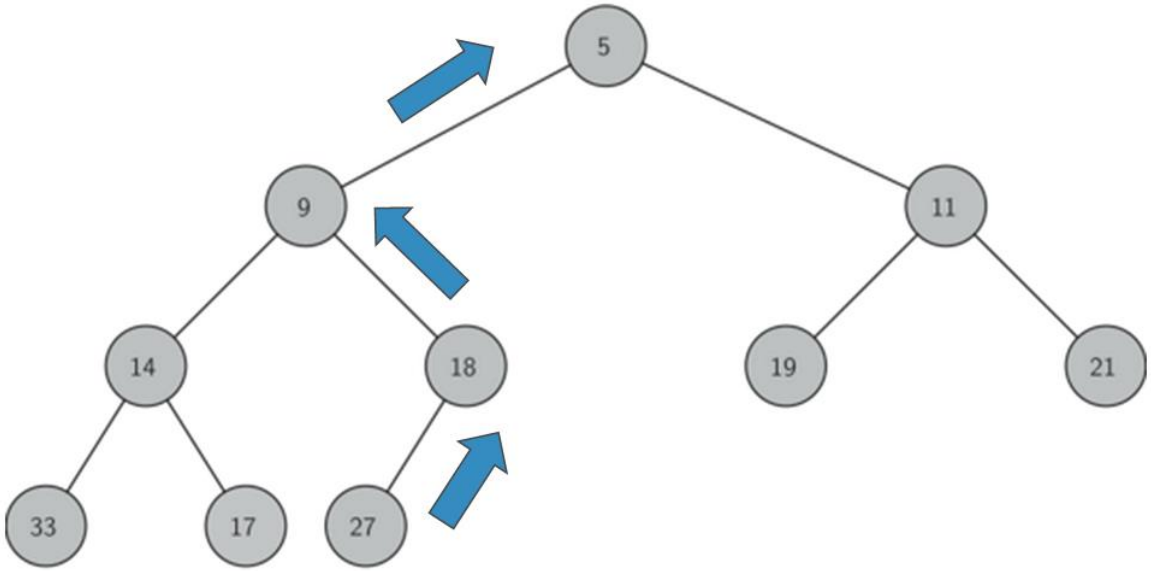
- 完全二叉树由于其特殊性，可以用非嵌套列表，以简单的方式实现，并具有很好的性质：
  - 如果节点在列表中的下标为p，那么其左子节点下标为2p，右子节点为2p+1
  - 如果节点在列表中下标为n，那么其父节点下标为n//2



0	1	2	3	4	5	6	7	8	9	10	11
0	5	9	11	14	18	19	21	33	17	27	

堆次序Heap Order

- 所谓“堆”的特性，是指堆中任何一个节点x，其父节点p中的key均小于等于x中的key。
  - 这样，符合“堆”性质的二叉树，其中任何一条路径，均是一个已排序数列
  - 符合“堆”性质的二叉树，其树根节点中的key最小



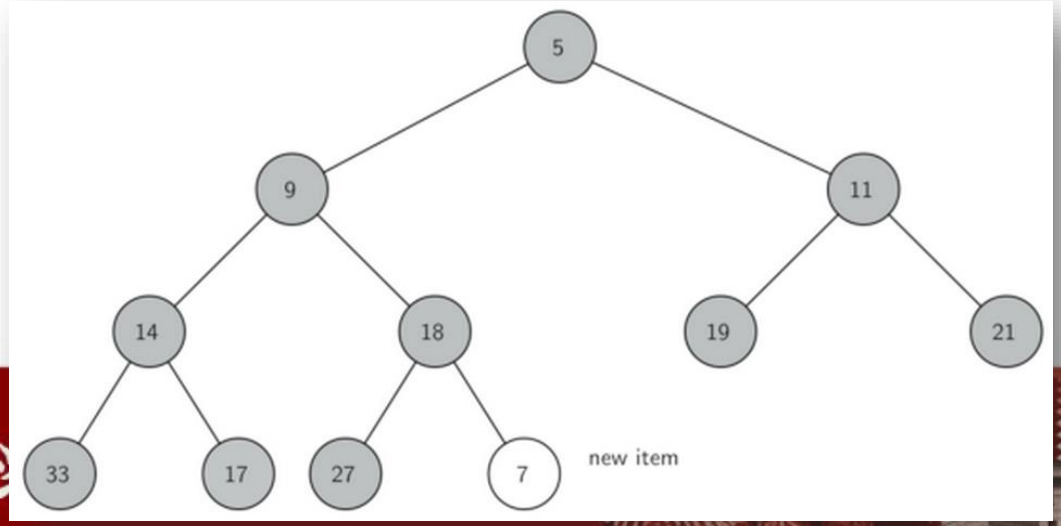
0	1	2	3	4	5	6	7	8	9	10	11
0	5	9	11	14	18	19	21	33	17	27	

二叉堆操作的实现

- 二叉堆初始化
  - 采用一个列表来保存堆数据，其中表首下标为0的项无用，但为了后面代码可以用到简单的整数乘除法，仍保留它。

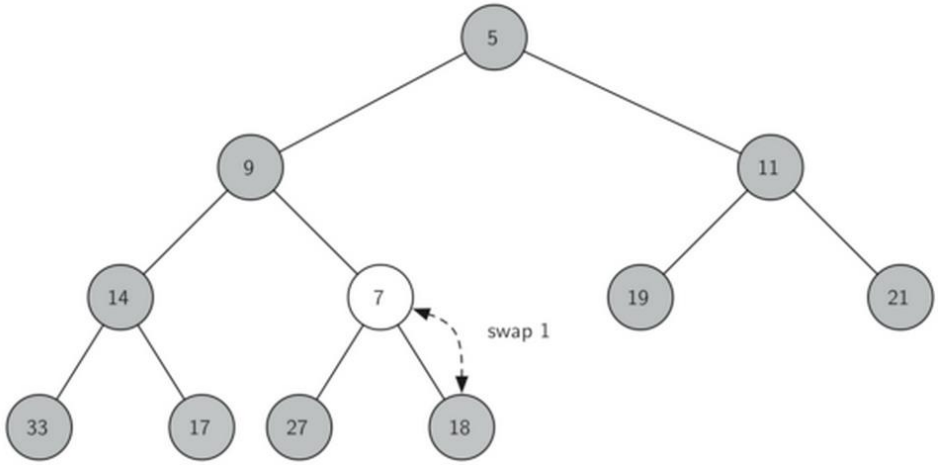
```
class BinHeap:
    def __init__(self):
        self.heapList = [0]
        self.currentSize = 0
```

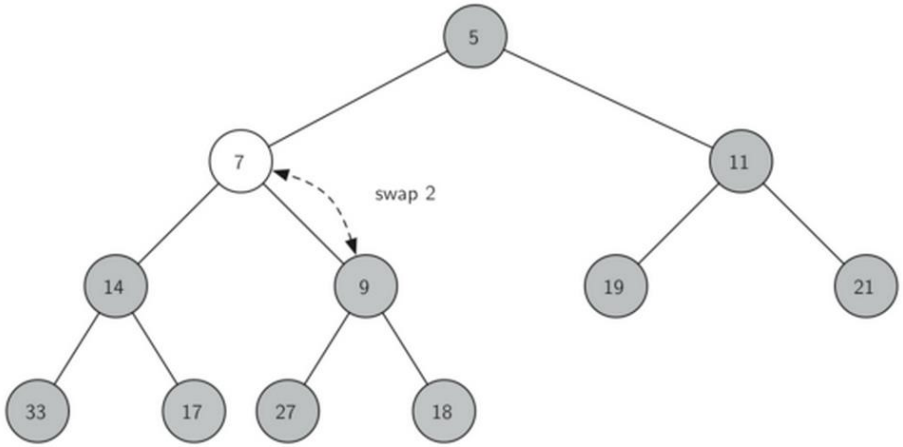
- insert(key)方法
  - 首先，为了保持“完全二叉树”的性质，新key应该添加到列表末尾



二叉堆操作的实现

- insert(key)方法
  - 但是，新key简单地加在列表末尾，显然无法保持“堆”次序
  - 虽然对其它路径的次序没有影响，但对于其到根的路径可能破坏次序
  - 于是需要将新key沿着路径来“上浮”到其正确位置
  - 注意：新key的“上浮”不会影响其它路径节点的“堆”次序





二叉堆操作的实现：insert代码

与父节点交换

沿路径向上

```
def percUp(self,i):
    while i // 2 > 0:
        if self.heapList[i] < self.heapList[i//2]:
            tmp = self.heapList[i // 2]
            self.heapList[i // 2] = self.heapList[i]
            self.heapList[i] = tmp
        i = i // 2
```

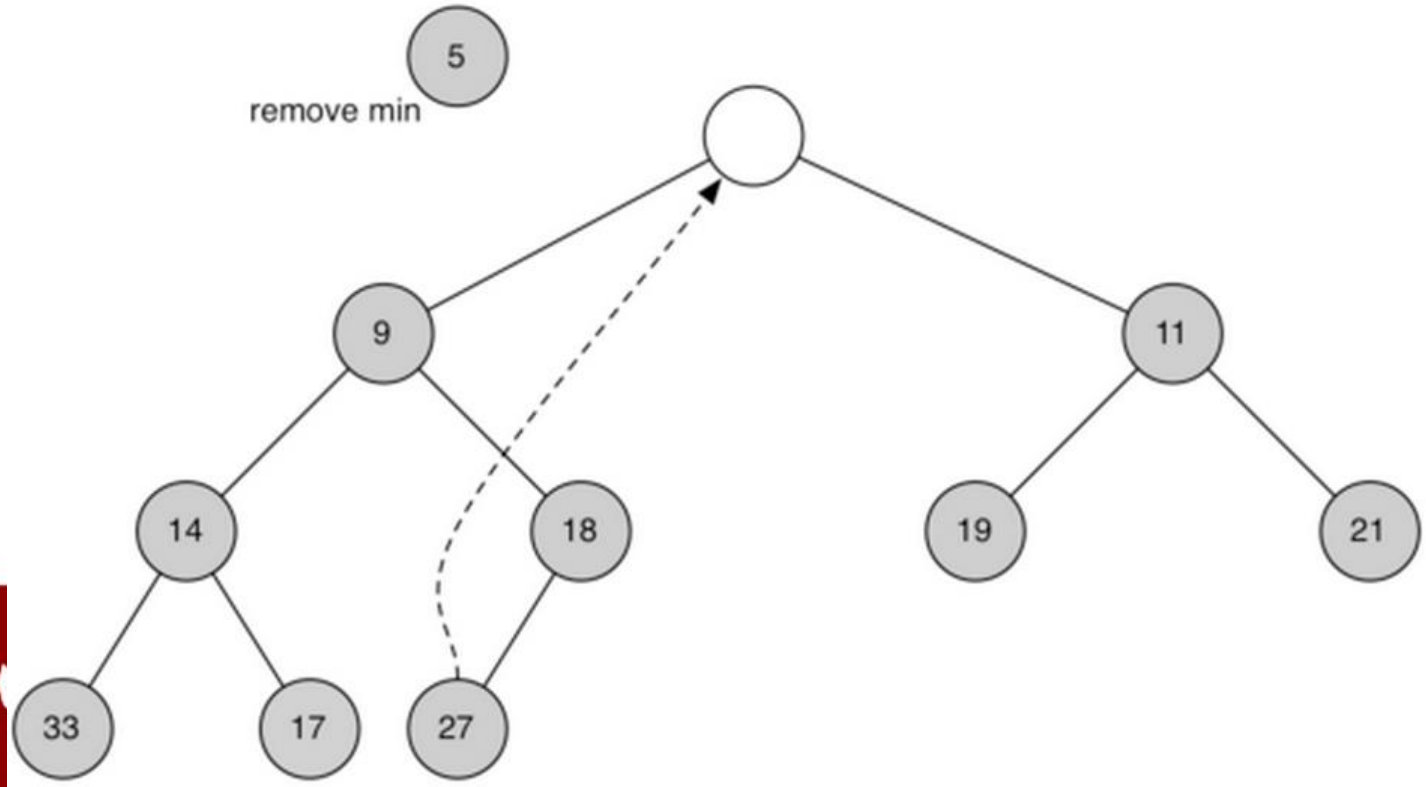
添加到末尾

新key上浮

```
def insert(self,k):
    self.heapList.append(k)
    self.currentSize = self.currentSize + 1
    self.percUp(self.currentSize)
```

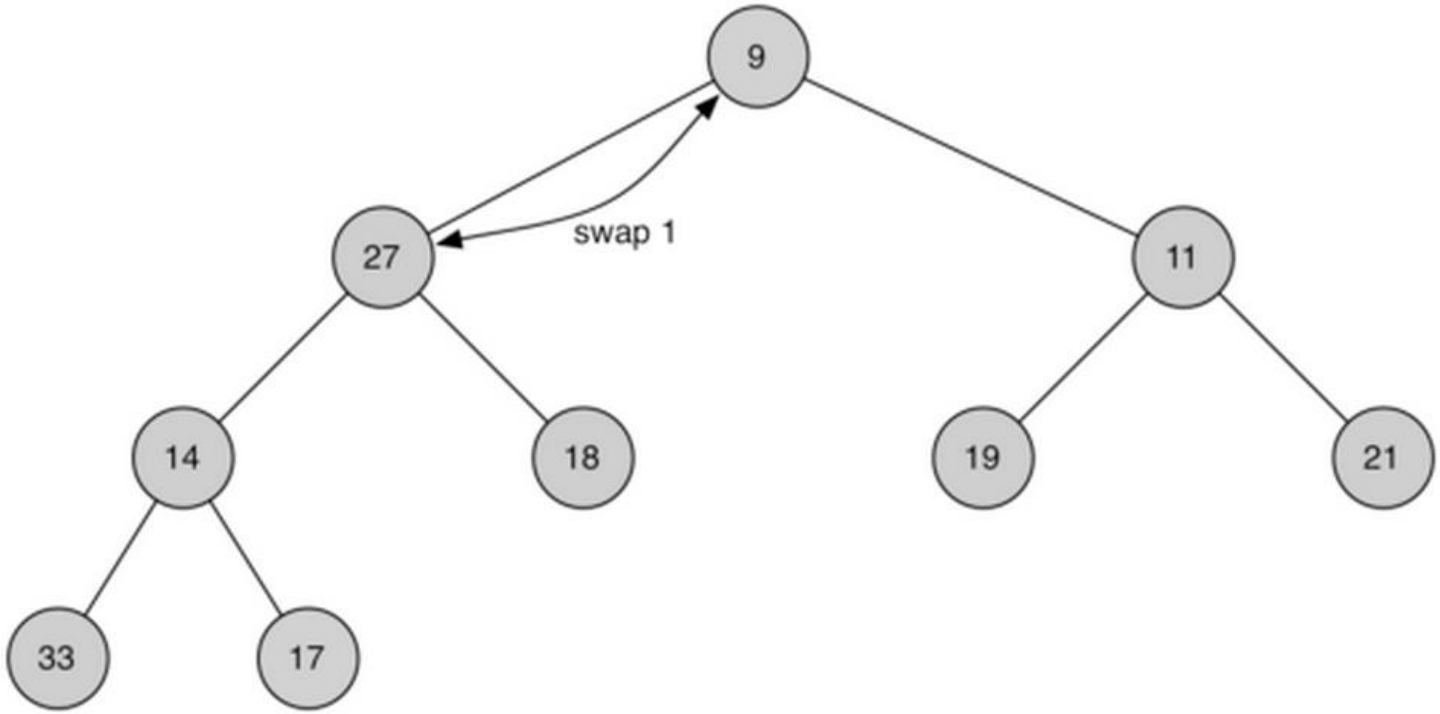
二叉堆操作的实现

- delMin()方法
  - 首先，移走整个堆中最小的key——位于堆首位的根节点heapList[1]
  - 为了保持“完全二叉树”的性质，用最后一个节点来代替根节点



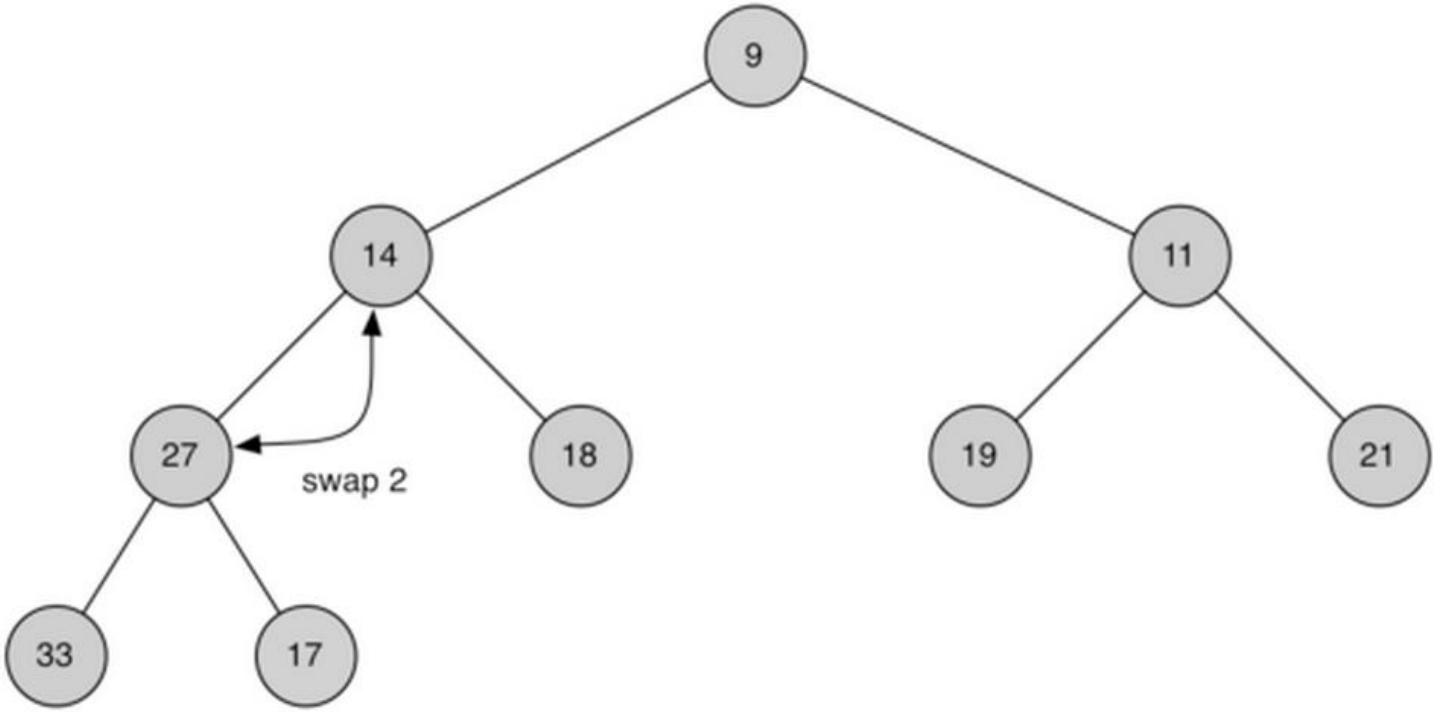
二叉堆操作的实现

- delMin()方法
  - 同样，这么简单的替换，还是破坏了“堆”次序
  - 解决方法：将新的根节点沿着一条路径“下沉”，直到比两个子节点都小
  - “下沉”路径的选择：如果比子节点大，那么选择较小的子节点交换下沉



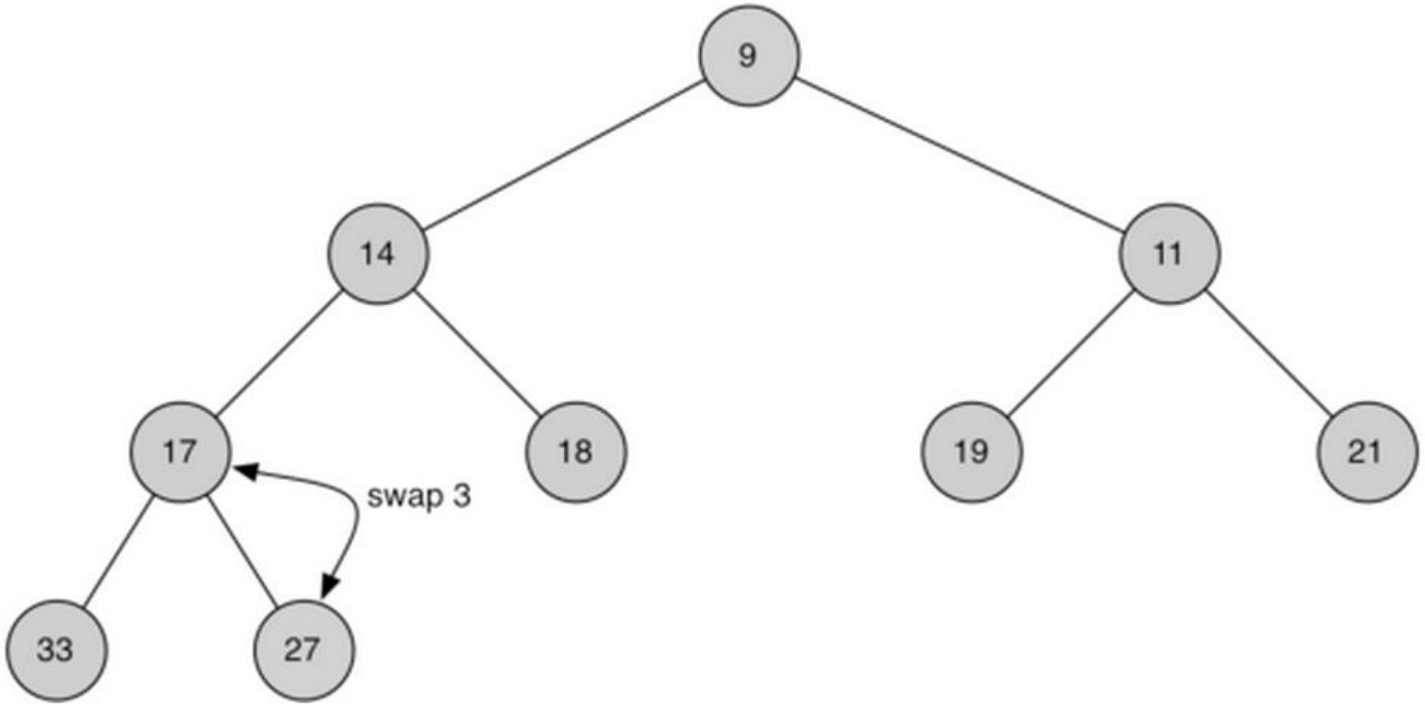
二叉堆操作的实现

- delMin()方法
  - 同样，这么简单的替换，还是破坏了“堆”次序
  - 解决方法：将新的根节点沿着一条路径“下沉”，直到比两个子节点都小
  - “下沉”路径的选择：如果比子节点大，那么选择较小的子节点交换下沉



二叉堆操作的实现

- delMin()方法
  - 同样，这么简单的替换，还是破坏了“堆”次序
  - 解决方法：将新的根节点沿着一条路径“下沉”，直到比两个子节点都小
  - “下沉”路径的选择：如果比子节点大，那么选择较小的子节点交换下沉



二叉堆操作的实现：delMin代码

交换下沉

沿路径向下

唯一子节点

返回较小的

移走堆顶

新顶下沉

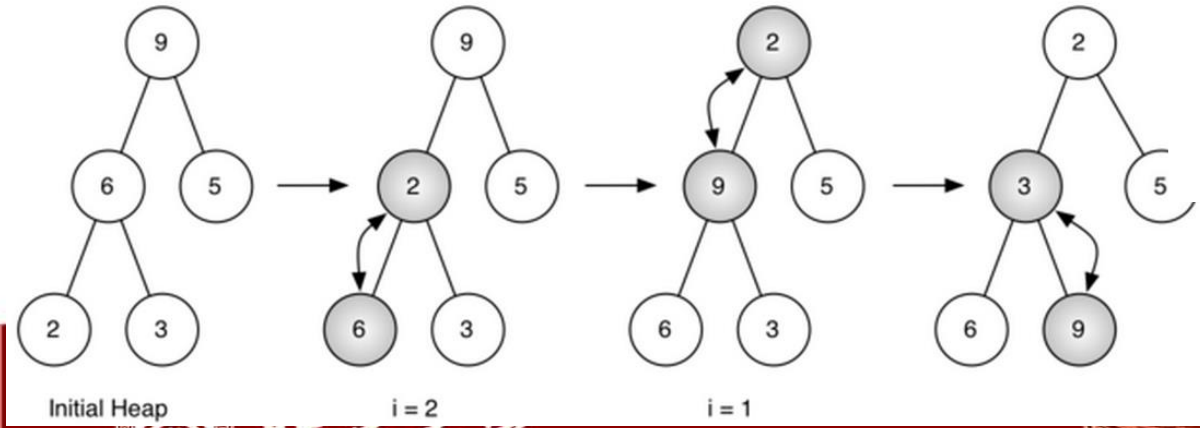
```
def percDown(self,i):
    while (i * 2) <= self.currentSize:
        mc = self.minChild(i)
        if self.heapList[i] > self.heapList[mc]:
            tmp = self.heapList[i]
            self.heapList[i] = self.heapList[mc]
            self.heapList[mc] = tmp
        i = mc

def minChild(self,i):
    if i * 2 + 1 > self.currentSize:
        return i * 2
    else:
        if self.heapList[i * 2] < self.heapList[i * 2 + 1]:
            return i * 2
        else:
            return i * 2 + 1

def delMin(self):
    retval = self.heapList[1]
    self.heapList[1] = self.heapList[self.currentSize]
    self.currentSize = self.currentSize - 1
    self.heapList.pop()
    self.percDown(1)
    return retval
```

二叉堆操作的实现

- buildHeap(lst)方法：从无序表生成“堆”
  - 我们最自然的想法是：用insert(key)方法，将无序表中的数据项逐个insert到堆中，但这么做的总代价是O(nlog n)
  - 其实，用“下沉”法，能够将总代价控制在O(n)

从最后节点的父节点开始
因叶节点无需下沉

```
def buildHeap(self,alist):
    i = len(alist) // 2
    self.currentSize = len(alist)
    self.heapList = [0] + alist[:]
    print(len(self.heapList), i)
    while (i > 0):
        print(self.heapList, i)
        self.percDown(i)
        i = i - 1
    print(self.heapList,i)
```



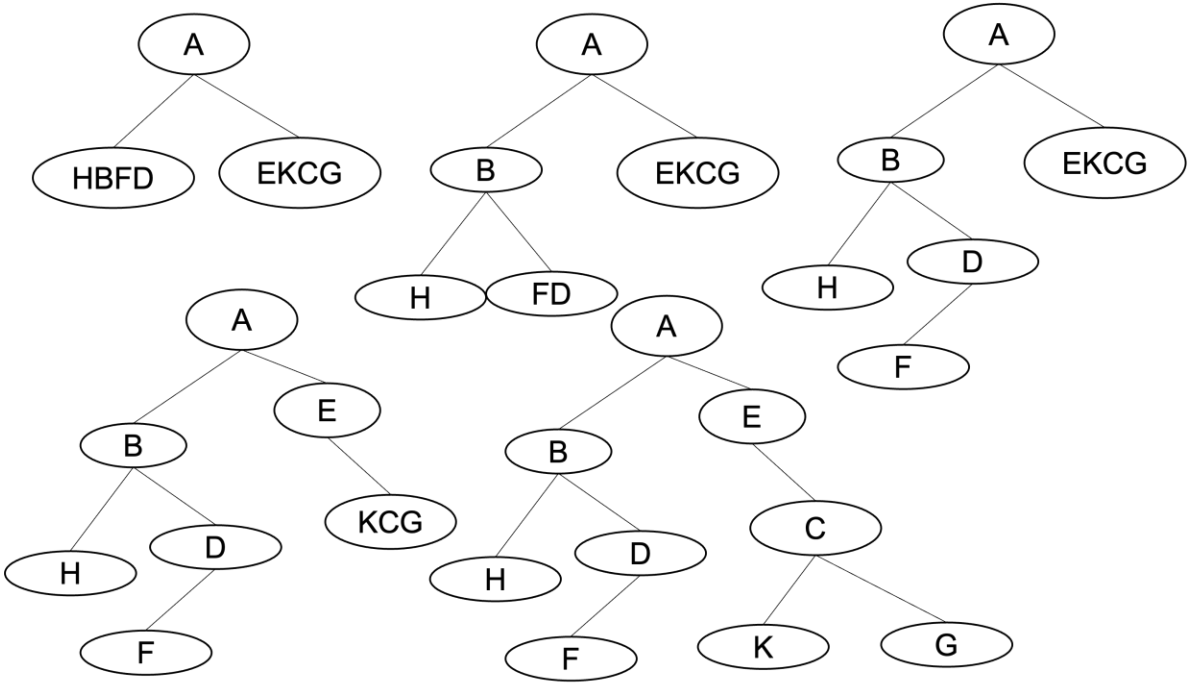
如何建立一棵二叉树?

思考

- 前、中、后序哪几种结合可以恢复二叉树的结构？
  - 已知某二叉树的前序序列为 {ABHDFECKG},
    中序序列为 {HBFDAEKCG}；
    则后序序列为 _____。
- 前序、中序、后序遍历结果中，所有叶节点都以相同的顺序出现吗？

利用中序、前序遍历生成二叉树

- 前序序列ABHDFECKG
- 中序序列HBFDAEKCG



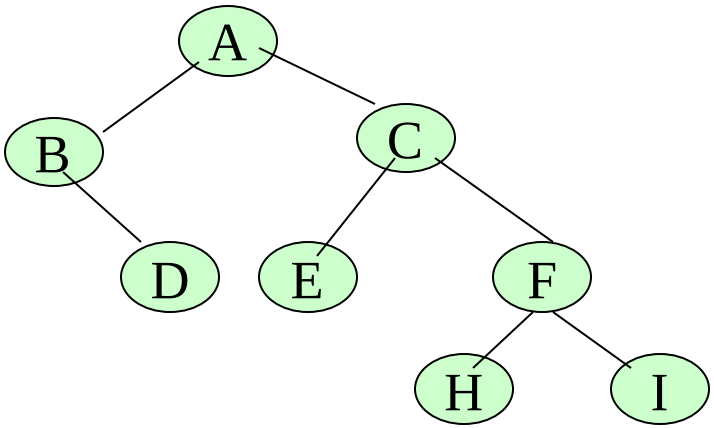
北京大学

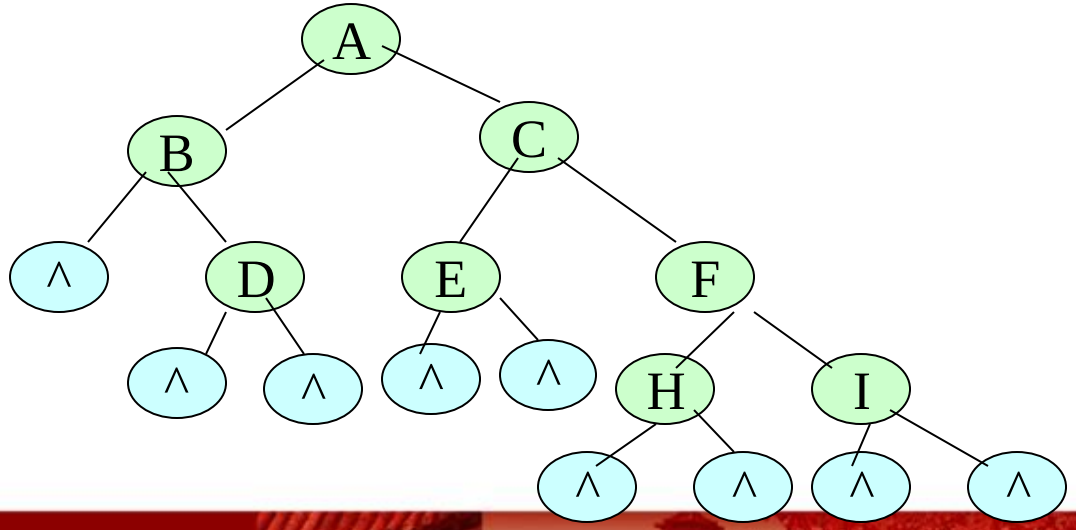
依据一个特殊数据序列建立二叉树

- 上述方法的困难：
  - 前序、中序、后序序列中任一个不能唯一确定一棵二叉树，所以不能直接用
  - 中序序列和前序序列 或 中序序列和后序序列 可以唯一确定，但是不方便
- 解决方法：读入一个特殊数据序列来建立二叉树
  - 将二叉树进行扩充，成为一棵扩充的二叉树，此时利用此扩充二叉树的前序/层次序列可以唯一确定其原二叉树

用扩充二叉树的前序列生成二叉树

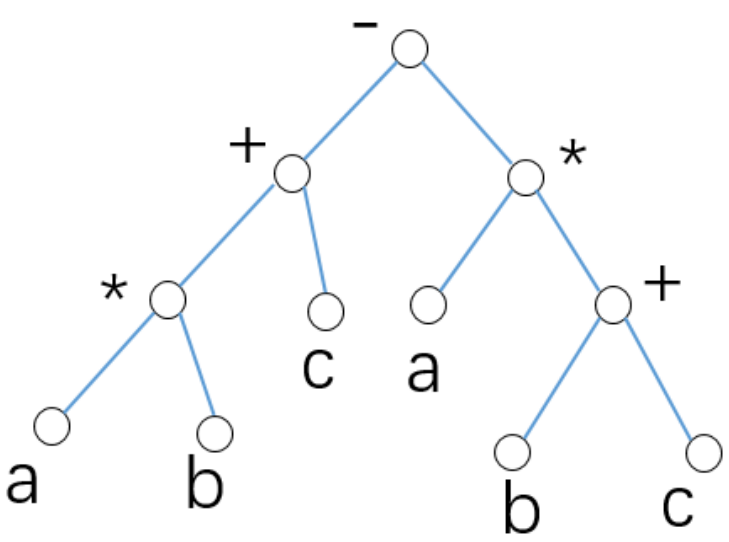
- ABDCEFHI>>>AB@D@@CE@@FH@@I@@
  - @表示为空节点
  - 设从A开始，A的左子树为B，B的左子树为空，因此D一定是B的右子节点
  - D有两个空子节点，说明C一定是A的右子节点





用扩充二叉树的前序列生成二叉树

- 例：利用扩充的前序序列建立二叉树
  - 读入A B D @ @ @ C E @ G @ @ F H @ @ I @ @,
  - 建立相应的llink-rlink表示法

表达式二叉树

- 前序（前缀）： - + * a b c * a + b c
- 中序 ： a * b + c - a * (b + c)
- 后序（后缀）：a b * c + a b c + * -